

Vision & Multimodal AI Engineer

The memorize-cold sheet: what you must be able to answer in your sleep

Built across the entire visual AI stack: geometry, classical filtering, CNNs, ViTs, object detection, segmentation, SSL, CLIP, modern VLMs, diffusion, flow matching, video foundation models, 3DGS, and edge vision systems. Interview Q&A first, derivations only where the answer is the derivation. Snapshot: October 2026.

How to use this sheet

- ▶ **Three passes.** Day 1: read a section. Day 3: cover every answer and state the math, mechanics, and trade-offs out loud. Day 7 and 21: drill only missed items.
- ▶ **Boxes.** **Blue** = interview Q&A. **Rust** = numbers, formulas, and invariants to memorize. **Green** = traps interviewers use to catch out candidates who memorize buzzwords without understanding the physics.
- ▶ **The five numbers of any vision model:** $H \times W$ (resolution), C (channels / embedding dimension), P (patch size / stride), L (layers / depth), and N_{tok} (visual tokens). FLOPs, receptive field, memory, and latency all follow.
- ▶ **Answer shape:** mechanism $\rightarrow$ number $\rightarrow$ trade-off $\rightarrow$ what you would measure and debug.
- ▶ **FLOPs vs MACs.** 1 MAC = 2 FLOPs. Conv formulas here count FLOPs ($2\times$); transformer formulas follow the Swin/ViT papers and count MACs ($4ND^2 + 2N^2D$ per attention layer). Say which one you mean in an interview.

THE 12 NUMBERS EVERYONE ASKS IN VISION

$2C_{\text{in}}C_{\text{out}}K^2HW$	FLOPs of a 2D convolution (H, W output spatial size)
$\frac{HW}{p^2}$	Visual tokens for an image in ViT ($P = 14 \implies 256$ for 224^2)
25.6M/4.1G	ResNet-50 parameters and FLOPs at 224×224
86M/307M	ViT-B/16 and ViT-L/16 parameters (17.6/61.6 GMACs at 224^2)
$8\times$	Spatial compression factor of standard SD / Flux VAEs ($512^2 \rightarrow 64^2$)
$\tau \approx 0.07$	CLIP initial temperature ($1/\tau \approx 14$); learned logit scale clamped at 100
59 floats	One 3D Gaussian: pos 3, quat 4, scale 3, opacity 1, SH 48 = 236 B in FP32
632M	SAM (Segment Anything) ViT-H image encoder parameter count
0.18215	SD 1.x VAE latent scale factor (SDXL: 0.13025); forget it and samples are grey mush
$\text{IoU} \geq 0.5$	Standard VOC/PASCAL positive detection match threshold
$RF_l = RF_{l-1} + (K-1)J$	Cumulative receptive field recurrence relation
33.3 ms	Real-time camera streaming latency budget at 30 FPS

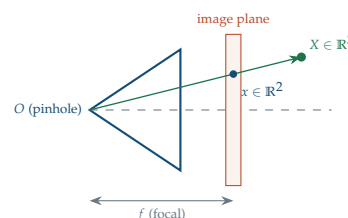
Contents

1	Geometry, Projection & Classical Vision	1
2	Classical Vision: Keypoints, Geometry & Flow	2
3	CNN Foundations, ResNet & Modern Convolutions	3
4	Modern ConvNets: MobileNet, ShuffleNet & ConvNeXt	3
5	Vision Transformers (ViT), Scaling & Tokenization	4
6	Hierarchical Vision Transformers: Swin & Hybrids	5
7	Object Detection: Two-Stage Detectors, FPN & mAP	5
8	Single-Stage Detectors: RetinaNet & The YOLO Lineage	6
9	Query-Based Detection: DETR, Deformable DETR & RT-DETR	7
10	Image & Video Segmentation: U-Net to SAM 2	8
11	Foundation Segmentation: SAM & SAM 2	8
12	Self-Supervised Vision: Contrastive, MAE & DINOv2	9
13	SSL Deep Dive: MAE, DINO & DINOv2	10
14	Vision-Language Foundations: CLIP to SigLIP	11
15	Multimodal LLMs: Architecture, AnyRes & Alignment	11

16	VLM Evaluation & Visual Hallucination	12
17	Generative Diffusion: DDPM, DDIM & Guidance	12
18	Latent Diffusion & Diffusion Transformers (DiT)	13
19	Flow Matching, Rectified Flow & Flux.1	14
20	Video Foundation Models: Spatio-Temporal DiT	15
21	3D Vision: NeRF to 3D Gaussian Splatting	15
22	3D Gaussian Splatting: Math, Rasterizer & SH	16
23	Vision Benchmarks, Metrics & Evaluation	17
24	Production Vision, Optimization & Edge Deployment	17
25	Vision System Design: Autonomous Driving to Visual RAG	18
26	Implement from Scratch: Vision Code You Write Blind	19
27	From-Scratch Vision Training Loops	19
28	Worked Napkin Problems in Vision	20
29	Vision Systems: Hardware Scaling & Napkin Math II	20
30	Debugging Playbooks: Vision Failure Modes	21
31	Rapid-Fire Q&A and Vision Glossary	22
31.1	Rapid-Fire Drills (Answer in under 5 seconds)	22
31.2	The Vision Glossary	22
32	High-Velocity Vision Interview Drill	22
33	Master Vision Formula Sheet	23
34	30 Seminal Vision Papers	23
35	References: The Vision Canon	24
36	The 21-Day Vision Mastery Curriculum	24

1 Geometry, Projection & Classical Vision

[epipolar geometry · pinhole optics · feature detectors · filtering · optical flow]



World to pixel coordinates. 3D world point $X_w = [X, Y, Z, 1]^T$ maps to 2D homogeneous pixel $x = [u, v, 1]^T$ via:

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K}[\mathbf{R} \mid \mathbf{t}] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

where $\mathbf{K}$ is the 3×3 intrinsic matrix, $\mathbf{R} \in \text{SO}(3)$ is rotation, and $\mathbf{t} \in \mathbb{R}^3$ is translation.

- ▶ Intrinsic matrix: f_x, f_y focal lengths, c_x, c_y principal point, s skew.
- ▶ Depth ambiguity: $\lambda = Z_c$ is lost in projection.

EPIPOLAR GEOMETRY & CORE TRANSFORMATIONS

Two calibrated cameras observing scene point X have image coordinates x, x' .

- ▶ **Essential Matrix** $\mathbf{E} = [\mathbf{t}]_{\times} \mathbf{R} \in \mathbb{R}^{3 \times 3}$: relates normalized camera coordinates: $x'^T \mathbf{E} x = 0$. $\mathbf{E}$ has 5 degrees of freedom (two equal singular values, one zero).
- ▶ **Fundamental Matrix** $\mathbf{F} = \mathbf{K}'^{-T} \mathbf{E} \mathbf{K}^{-1}$: relates uncalibrated pixel coordinates: $x'^T \mathbf{F} x = 0$. $\mathbf{F}$ has rank 2, 7 degrees of freedom. Epipolar line $l' = \mathbf{F}x$. Epipoles satisfy $\mathbf{F}e = 0, \mathbf{F}^T e' = 0$.
- ▶ **Normalized 8-Point Algorithm:** Hartley's algorithm translates point centroid to $(0, 0)$ and scales average distance to $\sqrt{2}$ before constructing matrix $Af = 0$. Enforces $\det(\mathbf{F}) = 0$ by zeroing smallest singular value in SVD.
- ▶ **Homography** $\mathbf{H} \in \mathbb{R}^{3 \times 3}$ (8 DoF): maps points on a 3D plane, or any

scene under pure camera rotation, between two views: $x' \sim Hx$. Solved with 4 point pairs using Direct Linear Transformation (DLT) + RANSAC.

INTERVIEW Q&A | CLASSICAL FEATURES, EDGES & KEYPOINTS

Q1. Walk through the 4 stages of Canny edge detection in detail.

1. Gaussian smoothing: Convolve image with Gaussian filter ($G_\sigma, \sigma \approx 1.4$) to suppress high-frequency sensor noise. **2. Gradient calculation:** Apply Sobel operators

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = G_x^T$$

to compute gradient magnitude $|G| = \sqrt{G_x^2 + G_y^2}$ and direction $\theta = \arctan(G_y/G_x)$. Quantize θ into 4 canonical sectors: $0^\circ, 45^\circ, 90^\circ, 135^\circ$.

3. Non-Maximum Suppression (NMS): Thin wide edges down to 1-pixel direction. At each pixel, compare $|G|$ with its two neighbors along direction θ . If $|G|$ is smaller than either neighbor, suppress it to 0. **4. Double thresholding & Hysteresis:** Define high threshold T_{high} and low threshold T_{low} ($T_{\text{high}} \approx 2 - 3 \times T_{\text{low}}$). Pixels with $|G| \geq T_{\text{high}}$ are marked as strong edges; pixels with $T_{\text{low}} \leq |G| < T_{\text{high}}$ are weak edges. Retain weak edges *only* if connected to a strong edge via 8-connectivity.

Q2. How does the Harris Corner Detector mathematically separate corners, edges, and flats?

Compute second moment matrix $M = \sum_{(u,v)} w(u,v) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$,

where $w(u,v)$ is a Gaussian window. The eigenvalues λ_1, λ_2 of M characterize the local intensity curvature:

- **Flat region:** $\lambda_1 \approx 0, \lambda_2 \approx 0$ (no intensity variation).
 - **Edge:** $\lambda_1 \gg \lambda_2 \approx 0$ (variation along gradient normal, invariant along tangent).
 - **Corner:** λ_1, λ_2 both large (variation in all directions).
- Harris avoids explicit eigendecomposition by defining corner response R :

$$R = \det(M) - k \cdot (\text{Tr}(M))^2 = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2 \quad (k \in [0.04, 0.06])$$

$R > 0$ indicates a corner, $R < 0$ indicates an edge, and $|R| \approx 0$ indicates a flat region.

Q3. Compare SIFT, SURF, and ORB feature extractors.

SIFT (Lowe, 2004): Scale-space DoG (Difference of Gaussians) extrema localization; 3D quadratic sub-pixel refinement; gradient orientation histogram assignment (rotation invariance); 4×4 spatial grid $\times 8$ orientation bins = 128-d descriptor. High accuracy, robust to illumination and 3D affine viewpoint shifts, but computationally slow. **SURF (Bay et al., 2006):** Replaces Gaussian second derivatives with box filters evaluated via Integral Images; Haar-wavelet responses; 64-d descriptor. $3 \times$ faster than SIFT. **ORB (Rublee et al., 2011):** FAST corner detector with intensity centroid orientation (oFAST) + steered BRIEF binary descriptors (rBRIEF). Matches descriptors via bitwise Hamming distance (XOR + popcount) on CPU. Runs $100 \times$ faster than SIFT, patent-free, widely used in real-time visual SLAM.

Q4. State the Optical Flow constraint equation and compare Lucas-Kanade vs Horn-Schunck.

Brightness Constancy Assumption: $I(x+u, y+v, t+1) = I(x, y, t)$. First-order Taylor expansion yields:

$$I_x u + I_y v + I_t = 0 \iff \nabla I \cdot \mathbf{v} + I_t = 0$$

This is 1 equation with 2 unknowns (u, v) (Aperture Problem: flow parallel to edges is ambiguous). **Lucas-Kanade:** Assumes flow is constant within a small local $n \times n$ window (typically 5×5). Solves the overdetermined system by least squares:

$$\begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}^{-1} \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

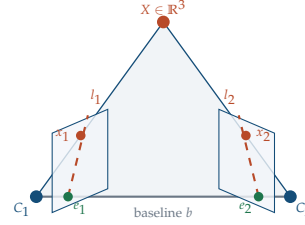
The 2×2 matrix is Harris's M : LK is reliable exactly where Harris finds corners (both eigenvalues large). Fails on large displacements (requires multi-scale image pyramids). **Horn-Schunck:** Global variational formulation minimizing brightness error + global smoothness: $E(u, v) = \iint ((I_x u + I_y v + I_t)^2 + \alpha^2 (\|\nabla u\|^2 + \|\nabla v\|^2)) dx dy$. Solves dense flow everywhere via Gauss-Seidel iteration, filling in smooth flow over untextured regions.

COORDINATE & GEOMETRIC TRAPS

- **Essential vs Fundamental Matrix:** E operates on normalized ray coordinates $x_{\text{norm}} = K^{-1}x_{\text{pix}}$ and depends purely on extrinsic pose $[R|t]$. F operates on raw pixel coordinates and absorbs camera intrinsics K .
- **Rank-2 Enforcement in Fundamental Matrix:** Raw linear estimation yields full rank-3 F . If SVD singular values $[\sigma_1, \sigma_2, \sigma_3]$ are not forced to $[\sigma_1, \sigma_2, 0]$, epipolar lines will fail to intersect at a unique epipole.

2 Classical Vision: Keypoints, Geometry & Flow

[Harris corner · SIFT · ORB · RANSAC · Essential/Fundamental · Lucas-Kanade · RAFT]



The Epipolar Constraint. X, C_1, C_2 define the epipolar plane. Baseline b pierces image planes at epipoles e_1, e_2 . Observing point x_1 constrains correspondence x_2 to lie along epipolar line $l_2 = Fx_1$.

- Calibrated: $x_2^T E x_1 = 0$ (5 DoF, SVD $[\sigma, \sigma, 0]$).
- Uncalibrated: $x_2^T F x_1 = 0$ (7 DoF, rank 2).
- Search: 2D correspondence reduced to 1D ray.

KEYPOINT DETECTORS & DESCRIPTORS AT A GLANCE

Method	Detector	Descriptor	Properties & Invariance
Harris	Structure Tensor M	None	Rotation, translation; NOT scale-invariant
SIFT	DoG Scale-Space	128-D L2	Scale, rotation, illumination; compute-heavy
SURF	Box filters	64-D L2	Fast scale-invariance via integral images
FAST	16-px circle test	None	Ultra-fast corner detector (~ 0.5 ms)
ORB	Oriented FAST	256-bit bin	Rotation-invariant; Hamming dist (POPCNT)

INTERVIEW Q&A | CLASSICAL CORNER DETECTION & INVARIANT DESCRIPTORS

Q5. How do you recover depth from a rectified stereo pair, and why does depth error grow as Z^2 ?

After rectification the epipolar lines are horizontal, so a point at depth Z appears at x_L and x_R on the same row with disparity $d = x_L - x_R$. Similar triangles give

$$Z = \frac{fB}{d} \quad (f \text{ in pixels, } B \text{ baseline in metres})$$

Error propagation: $\left| \frac{\partial Z}{\partial d} \right| = \frac{fB}{d^2} = \frac{Z^2}{fB} \implies \Delta Z \approx \frac{Z^2}{fB} \Delta d$.

Example: $f = 1000$ px, $B = 0.12$ m, matching error $\Delta d = 0.25$ px. At $Z = 10$ m, $\Delta Z \approx 0.21$ m; at $Z = 40$ m, $\Delta Z \approx 3.3$ m ($16 \times$ worse for $4 \times$ the range).

Levers: wider baseline (more occlusion, harder matching), longer focal length (narrower field of view), sub-pixel matching (SGM, RAFT-Stereo). Without rectification, **triangulate** by DLT: each view gives $x \times (PX) = 0$, two linear equations; stack and solve $AX = 0$ by SVD, then refine by minimizing reprojection error.

Q6. How does SIFT achieve scale invariance, and why does Lowe's Ratio Test reject false matches?

Scale Invariance: SIFT builds an octave scale space by convolving the image with Gaussians of increasing variance σ , then computes Difference-of-Gaussians (DoG):

$$D(x, y, \sigma) = (G(x, y, k\sigma) - G(x, y, \sigma)) * I(x, y)$$

Extrema (minima/maxima) are detected by comparing each pixel against its 8 neighbors in the current scale and $9 + 9 = 18$ neighbors in adjacent scales (26 comparisons). Sub-pixel localization fits a 3D quadratic Taylor polynomial.

Lowe's Ratio Test: When matching keypoint descriptor d_A against target set, let d_{B1} be the nearest neighbor (L_2 distance r_1) and d_{B2} be the second-nearest neighbor (L_2 distance r_2).

$$\text{Ratio} = \frac{r_1}{r_2} < \tau \quad (\tau \approx 0.75 - 0.80)$$

Ambiguous/repetitive features (e.g., foliage, building windows) produce $r_1 \approx r_2 \implies \text{Ratio} \approx 1.0$ and are discarded; unique salient features yield $r_1 \ll r_2$, ensuring high-confidence matching.

Q7. Derive the RANSAC iteration formula for geometric model fitting (e.g., Homography or Epipolar).

Given sample size s ($s = 4$ for Homography, $s = 8$ for Fundamental Matrix), outlier ratio $\epsilon \in (0, 1)$, and desired probability of success p (typically 0.99): The probability that a random draw of s points contains only inliers is $(1 - \epsilon)^s$. The probability that at least one of N random trials draws an all-inlier set is:

$$1 - (1 - (1 - \epsilon)^s)^N = p \implies N = \frac{\ln(1 - p)}{\ln(1 - (1 - \epsilon)^s)}$$

Example: For $\epsilon = 0.5$ (50% outliers) and Homography ($s = 4$): $(1 - \epsilon)^4 = (0.5)^4 = 0.0625$. For $p = 0.99$: $N = \frac{\ln(0.01)}{\ln(1 - 0.0625)} \approx \frac{-4.605}{-0.0645} \approx 72$ iterations.

INTERVIEW Q&A | EPIPOLAR GEOMETRY & OPTICAL FLOW

Q8. How does Zhang's camera calibration work, and what does PnP solve?

Zhang (2000): photograph a planar checkerboard ($Z = 0$) in ≥ 3 poses. Each view gives a homography $H = \lambda K[r_1 \ r_2 \ t]$. Because $r_1 \perp r_2$ and $\|r_1\| = \|r_2\|$, each view adds two linear constraints on the symmetric $B = K^{-T}K^{-1}$:

$$h_1^T B h_2 = 0, \quad h_1^T B h_1 = h_2^T B h_2$$

B has 6 unknowns up to scale, so 3 views suffice. Recover K from B (Cholesky), then refine K , lens distortion (k_1, k_2, p_1, p_2) and all poses with Levenberg-Marquardt on reprojection error (good calibration: < 0.3 px RMS).

PnP (Perspective-n-Point): given known K and n 3D-2D matches, recover the 6-DoF pose $[R \ | \ t]$. Minimal P3P uses 3 points (up to 4 solutions; a 4th point disambiguates); EPnP is $\mathcal{O}(n)$. Wrap in RANSAC, then refine by minimizing reprojection error. Used for AR anchoring, visual relocalization and robot grasping.

Q9. How does RAFT estimate optical flow, and why does it beat coarse-to-fine pyramids?

Classical flow (Q4) linearizes brightness constancy, so it only works for small motions and needs image pyramids, which blur away small fast objects. **RAFT (Teed & Deng, 2020):**

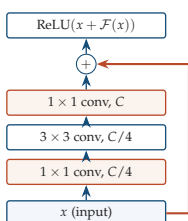
1. Extracts 1/8-resolution features $f(I_1), f(I_2)$ and builds a 4D **all-pairs correlation volume** $C_{ijkl} = \langle f_1(i, j), f_2(k, l) \rangle$, pooled into a 4-level pyramid over the last two dims.
2. A recurrent **ConvGRU** iteratively updates a single high-resolution flow field, $\mathbf{f}_{k+1} = \mathbf{f}_k + \Delta \mathbf{f}$, by looking up correlations around the current estimate (radius 4, all pyramid levels).
3. Loss: $\sum_k \gamma^{N-k} \|\mathbf{f}_{\text{gt}} - \mathbf{f}_k\|_1$ with $\gamma = 0.8$ over 12 training iterations. Large displacements are visible from step 0 through the coarse correlation levels, yet flow is never estimated at low resolution, so thin and fast objects survive. Cost: $\mathcal{O}((HW)^2)$ memory for the volume.

CLASSICAL CV PRODUCTION TRAPS

- **Raw 8-Point Algorithm Failure:** Unnormalized coordinates cause numerical instability ($x \sim 10^3 \implies x_1 x_2 \sim 10^6$ while constant is 1). Always apply Hartley's normalization: translate centroid to origin and scale average distance to $\sqrt{2}$.
- **Hamming vs Euclidean Distance:** ORB descriptors are binary strings; using Euclidean distance (L_2) produces nonsense. Always use Hamming distance via hardware POPCNT (a ~ b).

3 CNN Foundations, ResNet & Modern Convolutions

[receptive field · residual learning · depthwise separable · ConvNeXt · BatchNorm]



The ResNet Bottleneck. Stacks 1×1 compress $\rightarrow 3 \times 3$ spatial $\rightarrow 1 \times 1$ expand. The skip connection carries the identity signal directly, preventing gradient vanishing:

$$\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_L} \left(\mathbf{I} + \frac{\partial}{\partial x_l} \sum_{i=l}^{L-1} \mathcal{F}_i(x_i) \right)$$

Even if gradients through $\mathcal{F}$ vanish (≈ 0), the identity $\mathbf{I}$ guarantees non-zero backpropagation back to layer l .

CONVOLUTION ARITHMETIC FORMULAS

$$O = \left\lfloor \frac{I - K + 2P}{S} \right\rfloor + 1$$

Output spatial dimension (Input I , Kernel K , Pad P , Stride S)

$$\text{FLOPs} = 2C_{\text{in}}C_{\text{out}}K^2H_0W_0$$

Exact multiply-adds ($2 \times$ MACs) for standard Conv2D

$$RF_l = RF_{l-1} + (K_l - 1)J_{l-1}$$

Receptive field of layer l ; jump

$$\frac{1}{C_{\text{out}}} + \frac{1}{K^2}$$

$J_l = J_{l-1} \cdot S_l$ ($J_0 = 1$)

Computation ratio of

Depthwise-Separable conv vs Standard Conv

INTERVIEW Q&A | ARCHITECTURE EVOLUTION & NORMALIZATION DYNAMICS

Q10. Derive the backpropagation gradient of Conv2D with respect to input X and weights W .

Forward convolution: $Y_{\text{out},i,j} = \sum_{c_{\text{in}}} \sum_{p=0}^{K-1} \sum_{q=0}^{K-1} W_{\text{out},c_{\text{in}},p,q} \cdot X_{c_{\text{in}},i-s+p,j-s+q}$.

- **Gradient w.r.t Weights:** By chain rule:

$$\frac{\partial \mathcal{L}}{\partial W_{\text{out},c_{\text{in}},p,q}} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial Y_{\text{out},i,j}} \cdot X_{c_{\text{in}},i-s+p,j-s+q}$$

This is equivalent to a cross-correlation between the input activation X and the upstream gradient $\frac{\partial \mathcal{L}}{\partial Y}$.

- **Gradient w.r.t Input:**

$$\frac{\partial \mathcal{L}}{\partial X_{c_{\text{in}},m,n}} = \sum_{c_{\text{out}}} \sum_{p,q} \frac{\partial \mathcal{L}}{\partial Y_{c_{\text{out}},m-p,n-q}} \cdot W_{c_{\text{out}},c_{\text{in}},p,q}$$

This is equivalent to a full convolution of the upstream gradient with the **180-degree spatially flipped kernel** $W^{\text{rot}180}$ (padding with $K - 1$).

Q11. Why did VGG stack multiple 3×3 convolutions instead of one 7×7 ?

Three 3×3 convolutions stacked with stride 1 have an effective receptive field of $1 + 3 \times (3 - 1) = 7$, matching a single 7×7 kernel. **Parameter reduction:** $3 \cdot (3^2 C^2) = 27C^2$ vs $1 \cdot (7^2 C^2) = 49C^2$ (45% fewer weights). **Expressivity:** Inserts three non-linear activation functions (ReLUs) rather than one, allowing more expressive discriminative functions.

Q12. Why does BatchNorm fail on small batch sizes, and why is GroupNorm or SyncBN used?

BatchNorm normalizes across batch dimension $\mathcal{B}$: $\hat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$. In

object detection and segmentation, high image resolutions restrict batch size per GPU to $B \in [2, 4]$. At $B = 2$, mini-batch mean $\mu_{\mathcal{B}}$ and variance $\sigma_{\mathcal{B}}^2$ exhibit extreme stochastic noise: batch statistics fluctuate wildly from step to step, corrupting training convergence and degrading test accuracy by $> 5\%$. **Solutions:**

1. **SyncBatchNorm:** Synchronizes mean and variance across all GPUs in distributed training via all-reduce, effectively restoring large virtual batch size ($B_{\text{effective}} = B_{\text{local}} \times N_{\text{GPUs}}$).
2. **GroupNorm (Wu & He, 2018):** Divides channels into $G = 32$ groups and normalizes purely across channels and spatial pixels ($C/G, H, W$). Independent of batch size B , ensuring stable training even at $B = 1$.

Q13. Derive He (Kaiming) initialization. When do you use Xavier instead?

For $y_l = W_l x_l$ with fan-in n_l and i.i.d. zero-mean weights, $\text{Var}(y_l) = n_l \text{Var}(w_l) \mathbb{E}[x_l^2]$. ReLU zeroes half of a symmetric input, so $\mathbb{E}[x_l^2] = \frac{1}{2} \text{Var}(y_{l-1})$. Keeping the variance constant across depth requires

$$\frac{1}{2} n_l \text{Var}(w_l) = 1 \implies \text{Var}(w) = \frac{2}{n_{\text{in}}}, \quad n_{\text{in}} = C_{\text{in}} K^2 \text{ for a conv.}$$

Xavier/Glorot: $\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$, derived for tanh/linear units (balances forward and backward variance). Using it with ReLU halves the signal every layer: 2^{-30} after 30 layers.

Modern practice: residuals plus normalization make nets far less sensitive; zero-init the last BN γ of each residual block so it starts as identity (Goyal et al.); ViTs use truncated normal with std 0.02.

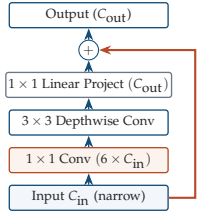
CONVOLUTIONAL GOTCHAS

- **Dilation Receptive Field:** Atrous/dilated conv with rate r has effective kernel $K' = K + (K - 1)(r - 1)$. For 3×3 with $r = 2$, $K' = 5$, growing receptive field without extra parameters.

- **Gridding artifacts:** Stacking consecutive atrous convolutions with identical dilation rate r creates "checkerboard" sampling holes. Fix: use hybrid dilated convolution (HDC) with coprime rates (e.g., $r = [1, 2, 5]$).

4 Modern ConvNets: MobileNet, ShuffleNet & ConvNeXt

[Depthwise separable · linear bottleneck · channel shuffle · compound scaling · ConvNeXt · GRN]



The Inverted Residual. MobileNetV2 expands to high dimension ($t = 6$) for depthwise filtering, then projects to low dimension with a linear bottleneck.

- Standard ResNet: High $\rightarrow$ Low $\rightarrow$ High.
- Inverted: Low $\rightarrow$ High ($6\times$) $\rightarrow$ Low.
- Linear Bottleneck: Non-linear ReLU in low-dim collapses the manifold; projection must be purely linear.

ARCHITECTURAL MILESTONES IN MODERN CONVOLUTIONS

Model	Key Op	Core Innovation	Savings
MobileNetV1	Depthwise Sep.	Factorizes spatial ($K \times K$) and channel (1×1)	$8 - 9\times$
MobileNetV2	Inverted Resid.	Low $\rightarrow$ Exp $6\times \rightarrow$ DW $\rightarrow$ Linear project (no ReLU)	Low MAC
ShuffleNetV2	Channel Shuffle	Equal channel width minimizes memory access cost (MAC)	Fast GPU
EfficientNet	Compound Scale	Jointly scales depth α , width β , and resolution γ	Pareto SOTA
ConvNeXt-V2	GRN + 7×7 DW	Global Response Normalization solves MAE feature collapse	Beats Swin

INTERVIEW Q&A | DEPTHWISE CONVOLUTIONS & BOTTLENECK DYNAMICS

Q14. Derive the exact FLOP reduction of Depthwise Separable Convolutions vs Standard Convolutions.

Let input feature map be $H \times W \times C_{in}$, output be $H \times W \times C_{out}$, and kernel size be $K \times K$:

- **Standard Convolution FLOPs:**

$$\text{FLOPs}_{\text{std}} = 2 \cdot H \cdot W \cdot C_{in} \cdot C_{out} \cdot K^2$$

- **Depthwise Separable Convolution FLOPs:** Composed of (1) Depthwise ($K \times K$ per channel) and (2) Pointwise (1×1 conv):

$$\text{FLOPs}_{\text{sep}} = 2 \cdot H \cdot W \cdot C_{in} \cdot K^2 + 2 \cdot H \cdot W \cdot C_{in} \cdot C_{out} \cdot 1^2$$

- **Theoretical Ratio:**

$$\frac{\text{FLOPs}_{\text{sep}}}{\text{FLOPs}_{\text{std}}} = \frac{C_{in}K^2 + C_{in}C_{out}}{C_{in}C_{out}K^2} = \frac{1}{C_{out}} + \frac{1}{K^2}$$

For standard 3×3 kernel ($K = 3$) and large $C_{out} \gg 1$:

$$\frac{\text{FLOPs}_{\text{sep}}}{\text{FLOPs}_{\text{std}}} \approx \frac{1}{9} \approx 0.111 \implies \sim 8 \text{ to } 9 \times \text{ FLOP reduction!}$$

Q15. Why does MobileNetV2 use Inverted Residuals with Linear Bottlenecks instead of classic ResNet bottlenecks?

Classic ResNet Bottleneck: High dimension $\rightarrow 1\times 1$ reduce (channel compression) $\rightarrow 3\times 3$ conv $\rightarrow 1\times 1$ expand. **MobileNetV2 Inverted Residual:** Low dimension (narrow) $\rightarrow 1\times 1$ expand ($6\times$ channels) $\rightarrow 3\times 3$ depthwise in high-dim $\rightarrow 1\times 1$ project (narrow).

- **Why expand before depthwise?:** Depthwise convolutions cannot change channel count; performing depthwise filtering in low-dimensional space severely restricts representation capacity. Expanding channels by $t = 6$ gives the depthwise filter an expressive manifold.
- **Why Linear Bottleneck (no ReLU at output)?:** Applying a non-linear activation (ReLU) on low-dimensional manifolds destroys information: any negative values are zeroed out, collapsing dimensions. In high-dimensional representations, manifold features remain intact after ReLU; in low-dim bottlenecks, the projection must remain purely linear.

Q16. Explain the ShuffleNet V2 design principles for practical inference latency on hardware.

Ma et al. showed that FLOPs do not correlate directly with wall-clock runtime; **Memory Access Cost (MAC)** dominates:

- Principle 1 (Equal channel width minimizes MAC):** For 1×1 conv with FLOPs $B = HWC_1C_2$, $\text{MAC} = HW(C_1 + C_2) + C_1C_2 \geq 2\sqrt{HWW} + \frac{B}{HWW}$. Equality holds when $C_1 = C_2$.
- Principle 2 (Excessive group convolution increases MAC):** Large group count g increases MAC for the same FLOP budget.
- Principle 3 (Network fragmentation reduces parallelism):** Multi-branch paths (e.g., Inception, NAS architectures) fragment GPU kernel launches and reduce device utilization.
- Principle 4 (Element-wise operations have high MAC/FLOP ratio):** Tensor addition, ReLU, and Squeeze-and-Excitation have negligible FLOPs but heavy memory round-trips.

INTERVIEW Q&A | CONVNEXT ARCHITECTURE & GLOBAL RESPONSE NORMALIZATION

Q17. How does ConvNeXt modernize standard ResNet-50 into a pure ConvNet competitive with Swin?

Liu et al. (2022) systematically modernized ResNet using ViT design principles:

- **Macro design:** ResNet stage ratio changed from (3,4,6,3) to (3,3,9,3) matching Swin.
- **Patchify stem:** Replaced 7×7 conv (stride 2) + maxpool with a single 4×4 stride-4 convolution patchifying stem.
- **Depthwise 7×7 conv:** Moved spatial conv before pointwise conv; enlarged kernel from 3×3 to 7×7 to match Swin's local window attention receptive field.
- **Inverted bottleneck:** Expanded channel width $4\times$ inside each block (like ViT MLP expansion ratio 4).
- **Activation & Normalization:** Replaced ReLU with GELU; reduced activations to 1 per block; replaced BatchNorm with LayerNorm.

Q18. What is Global Response Normalization (GRN) in ConvNeXt-V2, and why does it prevent feature collapse?

When training ConvNeXt with Masked Autoencoders (FCMAE), masked feature channels exhibit feature collapse (redundant/dead channels with near-identical activation maps). **GRN mechanism** computes spatial L_2 norm per channel, normalizes across channels, and recalibrates:

- Global feature aggregation:** $g_c = \|X_c\|_2 = \sqrt{\sum_{i,j} X_c(i,j)^2}$.
- Channel response normalization:** $n_c = \frac{g_c}{\frac{1}{C} \sum_{c'} g_{c'}}$.
- Feature calibration:** $X'_c = \gamma \cdot (X_c \odot n_c) + \beta + X_c$, initialized with $\gamma = 0, \beta = 0$.
GRN creates channel competition: active channels suppress redundant passive channels, preventing representation collapse in self-supervised pretraining.

MODERN CONVNET HARDWARE TRAPS

- **Depthwise Conv Slowdown on GPUs:** Depthwise convolutions have extremely low compute-to-memory ratio (arithmetic intensity). On GPUs without specialized Tensor Core depthwise acceleration, standard 3×3 dense convs often execute faster than depthwise convs despite higher theoretical FLOPs!
- **Fused 1×1 Convolutions:** Inverted bottleneck blocks should use kernel fusion ($1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$) in TensorRT/TorchScript to avoid multiple intermediate DRAM round-trips.

5 Vision Transformers (ViT), Scaling & Tokenization

[patch embedding · inductive bias · DeiT distillation · 2D PosEnc · NaViT patch packing]

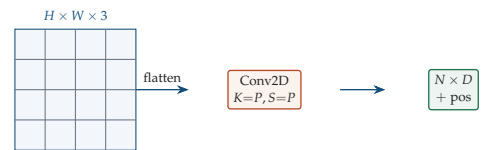


Image to Sequence. An image $x \in \mathbb{R}^{H \times W \times C}$ is split into $N = \frac{HW}{p^2}$ patches of size $P \times P$. Linear projection projects each patch to dimension D (equivalent to Conv2D with kernel P , stride P , D filters):

$$\mathbf{z}_0 = [\mathbf{x}_{cls}; \mathbf{x}_p^1 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{pos}$$

where $\mathbf{E} \in \mathbb{R}^{(P^2 \times C) \times D}$ and $\mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D}$.

STANDARD ViT MODEL CONFIGURATIONS & SCALING

Model	Layers	Hidden D	Heads	MLP Dim	Params (GMACs, 224 ²)
ViT-B/16	12	768	12	3072	86M (17.6)
ViT-L/16	24	1024	16	4096	307M (61.6)
ViT-H/14	32	1280	16	5120	632M (167.4)
ViT-g/14	40	1408	16	6144	1.0B (284.1)
ViT-22B	48	6144	48	24576	22B (~4.4k)

INTERVIEW Q&A | INDUCTIVE BIAS, DISTILLATION & TOKENIZATION

Q19. Why do CNNs outperform ViTs on small datasets, but lose at scale?

Inductive Biases: CNNs hardcode *translation equivariance* ($f(g(x)) = g(f(x))$) and *locality* (pixels close together interact first). This provides a massive data-efficiency inductive prior. **ViT:** Has almost zero 2D prior; self-attention is permutation-equivariant, and spatial structure must be learned entirely through E_{pos} and attention weights. **Empirical inflection point:** Under $\approx 10\text{M}$ images (ImageNet-1K), ResNet outperforms standard ViT. Above 100M images (ImageNet-21K, JFT-300M), ViT's higher capacity scales monotonically without saturating.

Q20. How does DeiT achieve high accuracy on pure ImageNet-1K?

Touvron et al. introduce **hard distillation**:

- Distillation Token:** Appends a learnable [DIST] token alongside [CLS]. The distillation token interacts with patches via self-attention and is supervised by a CNN teacher's hard label:

$$\mathcal{L} = \frac{1}{2} \mathcal{L}_{\text{CE}}(y, \sigma(z_{\text{cls}})) + \frac{1}{2} \mathcal{L}_{\text{CE}}(y_{\text{teacher}}, \sigma(z_{\text{dist}}))$$

- Heavy Regularization:** Mixup ($\alpha = 0.8$), CutMix ($\alpha = 1.0$), RandAugment, Stochastic Depth, and repeated augmentations prevent overfitting the unconstrained attention maps.

Reaches 85.2% top-1 on ImageNet-1K without external pretraining.

Q21. How do Mean Attention Distance and layer depth behave across a trained ViT?

Define Mean Attention Distance for head h at layer l :

$$\bar{D}_{l,h} = \sum_{j=1}^N A_{i,j}^{(l,h)} \cdot \|p_i - p_j\|_2$$

where p_i is the 2D spatial coordinate of patch i . **In shallow layers** ($l = 1 \dots 4$): Different heads specialize. Several heads have tiny attention distance ($\bar{D} \approx 2 - 4$ patches), acting exactly like localized 3×3 convolutional filters; other heads attend globally. **In deep layers** ($l = 8 \dots 12$): Attention distance is universally large across almost all heads ($\bar{D} \approx 10 - 14$ patches), performing global scene integration and semantic classification across the entire image.

Q22. How does NaViT (Patch n' Pack) resolve aspect ratio distortion and padding waste?

Standard ViTs resize images to fixed square shapes (224×224), causing non-square camera pictures (e.g., $16 : 9$) to stretch and distort geometric aspect ratios. Alternatively, padding with black bars wastes up to 50% of sequence FLOPs on empty border tokens. **NaViT (Dehghani et al., 2023):**

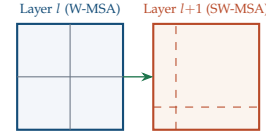
- Patch n' Pack:** Packs patches from multiple distinct images with arbitrary resolutions and aspect ratios into a single sequence of length L (e.g., $L = 4096$).
- Self-Attention Masking:** Uses an attention mask that forbids tokens from different images within the packed sequence from attending to each other.
- 2D Continuous Positional Encodings:** Queries and keys receive 2D fractional coordinates $(x/W, y/H) \in [0, 1]^2$. Eliminates all padding waste, preserves native image aspect ratios, and increases training throughput by $4 - 5\times$.

ViT ATTENTION TRAPS

- Quadratic token cost:** Doubling image resolution quadruples the token sequence length N , which increases self-attention computation and activation memory by $16\times$ ($\mathcal{O}(N^2)$).
- Bicubic vs Nearest for PosEnc:** Using nearest-neighbor interpolation causes high-frequency spatial artifacts and loss spikes during high-res fine-tuning. Bicubic maintains smooth positional phases.

6 Hierarchical Vision Transformers: Swin & Hybrids

[Swin Transformer · shifted windows · patch merging · cyclic masking · CoAtNet · FastViT]



Linear Complexity Vision. Standard global ViT self-attention scales quadratically: $\mathcal{O}(H^2W^2)$. Swin restricts self-attention to local $M \times M$ windows ($M = 7$):

$$\Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC$$

Linear in image area hw . To exchange cross-window information without global attention, consecutive blocks alternate between standard and **shifted** window partitions.

HIERARCHICAL FEATURE PYRAMID (SWIN VS ViT)

- Patch Merging:** Concatenates features of each 2×2 neighborhood (4C-dim), applies a linear layer to produce 2C-dim. Halves spatial resolution while doubling channels (produces $\frac{H}{4}, \frac{H}{8}, \frac{H}{16}, \frac{H}{32}$ pyramids compatible with FPN).
- Cyclic Shift & Masking:** Naive shifted window partitioning yields up to 2×2 extra partial windows at boundaries. Swin performs a cyclic shift to assemble partial windows into regular $M \times M$ windows, computes attention with a masking matrix $\mathbf{M}$ that blocks non-adjacent tokens, then reverse shifts.
- Relative Position Bias:** Adds bias matrix $\mathbf{B} \in \mathbb{R}^{M^2 \times M^2}$ from learned table $\hat{B} \in \mathbb{R}^{(2M-1) \times (2M-1)}$ based on coordinate offsets $(\Delta x, \Delta y) \in [-M+1, M-1]$.

INTERVIEW Q&A | DENSE PREDICTION, WINDOW ATTENTION & HYBRIDS

Q23. Why can't standard ViT directly replace ResNet in Faster R-CNN or U-Net?

1. Single-scale representation: ViT maintains constant token resolution ($\frac{H}{16} \times \frac{W}{16}$) throughout all depth. FPN and U-Net require multi-scale pyramids (C_2, C_3, C_4, C_5) to detect objects across small, medium, and large scales. **2. High-resolution memory explosion:** Object detection and segmentation require large inputs (1024×1024 or 1333×800). At $P = 16$, $1024^2 \implies 4096$ tokens, whose N^2 attention map takes $4096^2 \times 2$ bytes = 33.5 MB per head per layer: $\times 12$ heads $\times 24$ layers ≈ 9.6 GB of stored attention maps per image in training (without FlashAttention).

Q24. How does the Swin attention mask prevent cyclic leakage?

A cyclic shift wraps border tokens from the top-left to the bottom-right of the feature map, packing multiple disconnected image patches into the same $M \times M$ window. The attention mask $\mathbf{M}$ assigns $-\infty$ to logit entries (i, j) if tokens i and j originated from different sub-windows before the shift:

$$\text{Attention} = \text{Softmax} \left(\frac{QK^T}{\sqrt{d}} + \mathbf{B} + \mathbf{M} \right) V$$

$\text{Softmax}(\cdot)$ zeros out cross-patch attention weights, computing identical values to separate window evaluations in a single dense batched kernel without kernel launch overhead.

Q25. How does Pyramid Vision Transformer (PVT) achieve multi-scale representations without windows?

Wang et al. maintain global self-attention across the full image while reducing complexity via **Spatial-Reduction Attention (SRA)**: Instead of attending to all N tokens, keys and values are spatially downsampled by reduction ratio R_i using a strided convolution before projection:

$$\hat{K} = \text{Linear}(\text{Conv2D}(x, R_i)), \quad \hat{V} = \text{Linear}(\text{Conv2D}(x, R_i))$$

Reduces keys/values from N to N/R_i^2 . Complexity drops from $\mathcal{O}(N^2)$ to $\mathcal{O}(N^2/R_i^2)$, allowing global receptive fields in shallow stages with manageable memory.

Q26. What is CoAtNet's recipe for combining Convolutions and Attention?

Dai et al. organize blocks into 5 stages:

- Stage 0-1 (Conv):** Standard 3×3 conv and depthwise convs (captures low-level local edges and textures cheaply where spatial resolution is huge).
- Stage 2-3 (Attention + Conv):** Relative self-attention fused with depthwise convolution:

$$y = \text{Softmax} \left(\frac{QK^T}{\sqrt{d}} + W_{\text{conv}} + \mathbf{B} \right) V$$

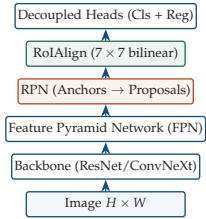
Balances inductive bias in shallow layers with global semantic reasoning in deep layers, achieving 86.0% top-1 on ImageNet-1K with fast training.

SWIN IMPLEMENTATION GOTCHAS

- **Dynamic image sizing in Swin:** every stage's feature map must tile into $M \times M$ windows. The last stage runs at stride 32, so clean inputs are multiples of $32 \cdot 7 = 224$; otherwise each stage pads to a window multiple and must crop back. Omitting pad/unpad logic silently shifts coordinates.
- **Batch dimension in Window Attention:** Folding spatial windows into batch ($B \cdot N_w, M^2, C$) requires memory contiguity (`.contiguous().view(...)`); failure to ensure contiguity triggers silent stride errors on CUDA.

7 Object Detection: Two-Stage Detectors, FPN & mAP

[Faster R-CNN · RPN anchors · RoIAlign · FPN · ClIoU Loss · COCO mAP]



The Object Detection Paradigm. Predicts class label $c \in \{1, \dots, K\}$ and coordinates $\mathbf{b} = [x_1, y_1, x_2, y_2]$ for all instances.

- **FPN:** Combines semantic richness of deep layers with high spatial resolution of shallow layers via top-down pathways and lateral 1×1 conv additions.
- **Decoupled Head:** Separate conv branches for classification (invariant to translation) and regression (sensitive to translation).

ROI OPERATIONS: POOL VS ALIGN

Operation	Coordinate Handling	Consequence
RoIPool (Fast R-CNN)	Rounds continuous RoI coordinates to feature stride ($x/16$), then rounds sub-cell bins.	Quantization error introduces severe sub-pixel spatial shifts; destroys mask AP.
RoIAlign (Mask R-CNN)	No quantization. Uses exact floating coordinates; samples 4 points per bin via bilinear interpolation .	Eliminates coordinate misalignment; gains +3.0 box AP and enables pixel-precise instance masks.

INTERVIEW Q&A | DETECTION MECHANICS, LOSSES & YOLO EVOLUTION

Q27. State the Complete IoU (CIoU) loss formulation and explain why it outperforms GIoU.

Standard GIoU penalizes empty area inside the smallest enclosing convex hull C : $\text{GIoU} = \text{IoU} - \frac{|C \setminus (A \cup B)|}{|C|}$. When one box is completely inside another, GIoU degenerates to standard IoU. **CIoU (Zheng et al., 2020)** incorporates 3 geometric factors simultaneously:

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{\text{gt}})}{c^2} + \alpha v$$

- $\frac{\rho^2(\mathbf{b}, \mathbf{b}^{\text{gt}})}{c^2}$: Normalized Euclidean distance between box center points, where c is diagonal length of the smallest enclosing box.
- $v = \frac{4}{\pi^2} \left(\arctan \frac{w^{\text{gt}}}{h^{\text{gt}}} - \arctan \frac{w}{h} \right)^2$: Measures aspect ratio consistency.
- $\alpha = \frac{v}{(1 - \text{IoU}) + v}$: Non-overlapping balance parameter giving high priority to aspect ratio when IoU is high. Accelerates convergence and improves localization accuracy across tight multi-scale boxes.

Q28. How does Faster R-CNN's Region Proposal Network generate and train proposals?

A 3×3 conv slides over each FPN level; every location holds k anchors (aspect ratios $\{1:2, 1:1, 2:1\}$, one scale per level, areas $32^2 \rightarrow 512^2$). Two sibling 1×1 heads output k objectness logits and $4k$ box deltas.

Box encoding relative to anchor (x_a, y_a, w_a, h_a) :

$$t_x = \frac{x - x_a}{w_a}, t_y = \frac{y - y_a}{h_a}, t_w = \log \frac{w}{w_a}, t_h = \log \frac{h}{h_a}$$

Labels: positive if $\text{IoU} > 0.7$ with a GT (or the best anchor for that GT), negative if < 0.3 , otherwise ignored. Sample 256 anchors per image at up to 1:1 pos:neg; loss is BCE + Smooth-L1 on positives.

Inference: decode, keep the top ~ 2000 per level, NMS at 0.7, keep

1000 proposals $\rightarrow$ RoIAlign $\rightarrow$ second stage (512 RoIs/image in training, 25% positives at $\text{IoU} \geq 0.5$).

Q29. How does FPN build its pyramid, and which level does an RoI read from?

The backbone gives $C_2 \dots C_5$ (strides 4 to 32). Top-down: $P_5 = \text{conv}_{1 \times 1}(C_5)$, then $P_l = \text{Up}_{2 \times}(P_{l+1}) + \text{conv}_{1 \times 1}(C_l)$, followed by a 3×3 conv per level to reduce upsampling aliasing. All levels have 256 channels, so every level carries strong semantics at its native resolution.

RoI-to-level rule: $k = \lfloor k_0 + \log_2(\sqrt{wh}/224) \rfloor$, $k_0 = 4$, clamped to $[2, 5]$. A 224^2 RoI reads P_4 , a 112^2 RoI reads P_3 : small objects use fine maps, large objects use coarse semantic maps.

Q30. How is COCO mAP actually computed?

Per class and IoU threshold: sort every detection in the dataset by score; match each greedily to the highest-IoU unmatched GT (crowd regions ignored). A match above the threshold is a TP, anything else an FP. Sweeping the score gives the PR curve; make precision monotone, $p(r) = \max_{r' \geq r} p(r')$, and average it at 101 recall points $\{0, 0.01, \dots, 1\}$: that is AP.

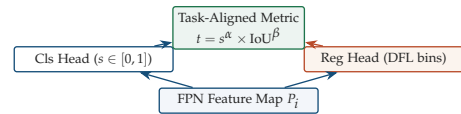
mAP = mean over 80 classes and 10 thresholds 0.50:0.05:0.95. $\text{AP}_S / \text{AP}_M / \text{AP}_L$ split by area $< 32^2, 32^2 - 96^2, > 96^2$. At most 100 detections per image are scored, which is why evaluation uses a 0.001 score threshold.

DETECTION METRIC TRAPS

- **COCO mAP vs VOC AP:** VOC 2007 measures AP at a single threshold $\text{IoU} = 0.50$ (mAP_{50}). COCO reports average AP over 10 thresholds: $[0.50 : 0.05 : 0.95]$ ($\text{mAP}_{50:95}$). A model can achieve 85% on mAP_{50} but drop to 55% on $\text{mAP}_{50:95}$ due to loose box boundary localization.
- **Bounding box parametrization:** (x, y, w, h) center vs (x_1, y_1, x_2, y_2) corner format. Mixing them up produces negative width/height in regression branches.

8 Single-Stage Detectors: RetinaNet & The YOLO Lineage

[SSD · Focal Loss · YOLOv1-v11 evolution · Anchor-free · TAL assigner · DFL · NMS-free YOLOv10]

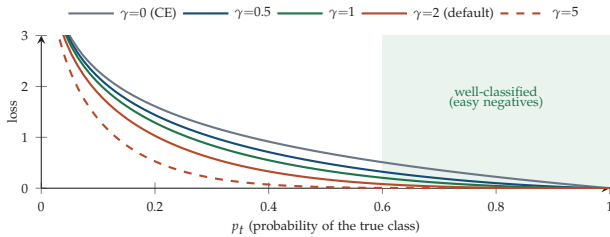


The Decoupled Anchor-Free Head. Modern YOLO (v8-v11) splits classification from localization.

- **TAL Alignment:** Top- k candidates ranked by joint metric $t = s^\alpha \times \text{IoU}^\beta$ to eliminate misalignment.
- **DFL Head:** Predicts continuous box boundaries as probability distributions across 16 discrete distance bins.

THE FULL YOLO EVOLUTION TIMELINE (V1 TO V11)

Model	Head Type	Core Architectural Innovations	NMS / Post
YOLOv1	Coupled Grid	First real-time detector (7 × 7 grid, FC regression)	Grid ownership
YOLOv3	Anchor-based	Darknet-53 + FPN multi-scale heads (P3, P4, P5)	K-means IoU
YOLOv4/5	Anchor-based	CSPDarknet backbone + PANet neck + Mosaic augment	IoU threshold
YOLOv7	Anchor-based	E-ELAN + RepConv structural re-parameterization	Lead assigner
YOLOv8	Anchor-free	Decoupled head + C2f block + Distribution Focal Loss	TAL Assigner
YOLOv9	Anchor-free	Programmable Gradient Information (PGI) + GELAN	Aux matching
YOLOv10	Dual-head	Dual assignments: 1-to-many train, 1-to-1 infer	Zero NMS ms
YOLOv11	Anchor-free	C3k2 blocks + C2PSA attention; DW-conv cls head	TAL + DFL



Focal loss $-(1 - p_t)^\gamma \log p_t$: raising γ flattens the loss of confident predictions toward zero.

INTERVIEW Q&A | FOCAL LOSS & LABEL ASSIGNMENT STRATEGIES

Q31. Derive Focal Loss and mathematically demonstrate why it prevents easy negative gradients from swamping training.

Standard binary cross entropy (CE) loss is $CE(p_t) = -\log(p_t)$, where $p_t = p$ if $y = 1$ and $p_t = 1 - p$ if $y = 0$. In dense single-stage detection, over 100,000 candidate anchors exist, with $\sim 99.9\%$ being easy background negatives ($p_t \approx 0.99$). Although $CE(0.99) \approx 0.01$ is small, summing over 10^5 candidates yields a gradient magnitude that completely overwhelms the few rare foreground positive targets ($p_t \approx 0.1$).

Focal Loss Formulation:

$$FL(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t) \quad (\text{typically } \gamma = 2.0, \alpha = 0.25)$$

Gradient Dynamics: For a positive ($y = 1, p_t = p = \sigma(z)$) the gradient with respect to the logit is:

$$\frac{\partial FL}{\partial z} = \alpha (1 - p_t)^\gamma (\gamma p_t \log p_t + p_t - 1)$$

($\gamma = 0$ recovers the CE gradient $p_t - 1$; negatives are symmetric with $p_t = 1 - p_t$). For an easy background sample ($p_t = 0.99$): The modulating factor is $(1 - 0.99)^\gamma = (0.01)^\gamma = 0.0001$ — a **10,000× gradient suppression!** For a hard positive or misclassified sample ($p_t = 0.2$): $(1 - 0.2)^\gamma = 0.64$ — virtually unaffected. The loss automatically focuses learning on hard informative candidates.

Q32. Compare Anchor-based vs Anchor-free detection: why did modern YOLO architectures abandon anchors?

- **Flaws of Anchor-based:** Requires manual heuristic tuning of anchor aspect ratios and scales (often dataset-specific k-means); dense spatial tiling creates hundreds of thousands of anchors, 99% of which are wasted; fixed IoU thresholds (e.g., 0.5) for positive matching fail on extreme-aspect objects.
- **Anchor-free Paradigm:** Predicts distance offsets from a grid point (x, y) directly to the 4 box boundaries (l, t, r, b) , eliminating hyper-parameter tuning and dramatically simplifying feature heads.

Q33. How does Task-Aligned Assigner (TAL) in YOLOv8/v11 prevent the misalignment between classification and localization?

Traditional assigners treat classification and regression independently: an anchor might have high classification score but poor IoU, or high IoU but low confidence. When Non-Maximum Suppression (NMS) ranks predictions solely by classification score, it retains boxes with poor spatial overlap and suppresses well-localized boxes.

TAL Metric: Computes a joint alignment metric t for each candidate anchor i and ground truth j :

$$t = s^\alpha \times \text{IoU}^\beta \quad (\text{default: } \alpha = 0.5, \beta = 6.0)$$

where s is the predicted class probability and IoU is the predicted box overlap with ground truth. Top- k candidates with the highest alignment metric t are assigned as positive samples. Furthermore, the classification target is dynamically scaled to t rather than a binary $\{0, 1\}$, forcing high classification scores to strictly coincide with high IoU accuracy.

INTERVIEW Q&A | DFL & NMS-FREE END-TO-END YOLOV10

Q34. What is Distribution Focal Loss (DFL) and why does it improve boundary precision?

Standard detectors predict a single deterministic scalar for bounding box offsets: $y \in \mathbb{R}$. However, in complex real-world scenes with blurry or occluded edges, the true boundary is inherently uncertain.

DFL Formulation: Predicts a discrete probability distribution over n intervals $\{y_0, y_1, \dots, y_n\}$ (e.g., 16 bins):

$$\hat{y} = \sum_{i=0}^n \text{softmax}(S_i) \cdot y_i$$

DFL minimizes the cross-entropy between the predicted distribution and the two discrete bins $[y_i, y_{i+1}]$ that flank the continuous ground truth y :

$$\mathcal{L}_{DFL}(S_i, S_{i+1}) = -((y_{i+1} - y) \log(S_i) + (y - y_i) \log(S_{i+1}))$$

This enables the network to express spatial uncertainty (sharp peak = clear edge; wide flat distribution = blurry/occluded boundary).

Q35. How does YOLOv10 eliminate NMS post-processing while maintaining high training accuracy?

The Dilemma: One-to-one matching (Hungarian assignment) enables NMS-free inference (like DETR) but suffers from slow training convergence due to sparse supervision. One-to-many matching provides rich supervisory gradients but produces duplicate boxes requiring NMS.

YOLOv10 Dual-Label Assignment:

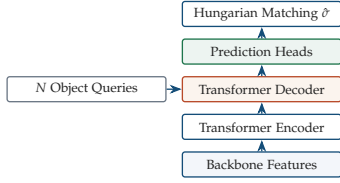
- **Training:** Evaluates two parallel heads sharing the same backbone/-neck: (1) a one-to-many head (optimizing gradient flow and representation learning) and (2) a one-to-one head (optimizing direct, unique prediction).
- **Harmonized Matching:** Adjusts metric alignment so both heads learn mutually consistent representations.
- **Inference:** Discards the one-to-many head completely! Runs only the one-to-one head, yielding zero-duplicate predictions with **0 ms post-processing latency**.

YOLO PRODUCTION TRAPS

- **Input Resolution Aspect Ratio Distortion:** Directly resizing images of arbitrary aspect ratios (e.g. 1920×1080) to square 640×640 distorts spatial aspect ratios, degrading small object recall. Always use letterboxing with gray padding, and ensure padding is aligned to a stride multiple of 32.
- **NMS Confidence Threshold Tuning:** Setting NMS confidence threshold too high (e.g. > 0.5) severely kills mAP on small objects. Standard COCO evaluation uses confidence threshold 0.001 with NMS IoU threshold 0.65.

9 Query-Based Detection: DETR, Deformable DETR & RT-DETR

[DETR · Hungarian matching · bipartite set loss · Deformable Attention · DINO-DETR · RT-DETR]



Direct Set Prediction. Bypasses hand-designed anchor boxes and greedy NMS entirely. Formulates detection as predicting an unordered set of N bounding boxes (e.g., $N = 300$).

- ▶ One-to-one matching enforces that each ground-truth object is assigned to exactly one query.
- ▶ No duplicate predictions $\implies$ zero NMS post-processing needed.

BIPARTITE MATCHING LOSS FORMULATION

Ground-truth objects padded with "no object" $\emptyset$ to size N : $y = (c_i, b_i)_{i=1}^N$. Predictions: $\hat{y} = (\hat{c}_{\sigma(i)}, \hat{b}_{\sigma(i)})_{i=1}^N$.

1. **Hungarian Matcher:** Finds optimal permutation $\hat{\sigma} \in \mathfrak{S}_N$ minimizing pairwise cost:

$$\hat{\sigma} = \arg \min_{\sigma \in \mathfrak{S}_N} \sum_{i=1}^N \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$$

where $\mathcal{L}_{\text{match}} = -\mathbf{1}_{\{c_i \neq \emptyset\}} \hat{p}_{\sigma(i)}(c_i) + \mathbf{1}_{\{c_i \neq \emptyset\}} (\lambda_{L1} \|b_i - \hat{b}_{\sigma(i)}\|_1 + \lambda_{\text{giou}} \mathcal{L}_{\text{giou}}(b_i, \hat{b}_{\sigma(i)}))$.

2. **Set Loss:** Computes cross-entropy and box regression loss along the optimal matching $\hat{\sigma}$.

INTERVIEW Q&A | DETR DYNAMICS, DEFORMABLE ATTENTION & REAL-TIME VARIANTS

Q36. Why did original DETR require 500 epochs to train, and how did Deformable DETR solve it?

The Bottleneck: Vanilla transformer attention computes dense softmax over all HW spatial tokens for every query. Early in training, attention weights are nearly uniform across the entire image: queries do not know where to look, making bipartite matching assignments unstable and erratic from step to step. **Deformable Attention:** Constrains attention to only K sampling points ($K = 4$) per head around a predicted 2D reference point $p_q = (p_{qx}, p_{qy}) \in [0, 1]^2$:

$$\text{DeformAttn}(z_q, p_q, x) = \sum_{m=1}^M W_m \left[\sum_{k=1}^K A_{mqk} \cdot W'_m x(p_q + \Delta p_{mqk}) \right]$$

where Δp_{mqk} are 2D offsets predicted from query z_q , and $x(\cdot)$ samples features via bilinear interpolation. Reduces complexity from $\mathcal{O}((HW)^2)$ to $\mathcal{O}(N_q \cdot K)$, enables multi-scale feature maps (C_3, C_4, C_5), and converges in **50 epochs** (10× faster).

Q37. What is Denoising Training in DINO-DETR?

Bipartite matching exhibits instability because small parameter updates flip which query is matched to which ground truth. DINO-DETR introduces **Contrastive Denoising (CDN)**:

- ▶ Adds noise to ground truth boxes: positive noisy boxes (small jitter within GT) and negative noisy boxes (large jitter).
 - ▶ Feeds noisy boxes directly as queries into the decoder with an attention mask preventing information leak to normal learned queries.
 - ▶ Forces the decoder to reconstruct the original box for positives and predict "background" for negatives.
- Acts as anchor priors during early training, bypassing matching instability and pushing detection performance to 63.3 mAP on COCO.

Q38. How does RT-DETR achieve real-time latency matching YOLOv8 on GPU?

Ly et al. identify that the multi-scale transformer encoder consumes 49% of FLOPs in Deformable DETR. **RT-DETR (Real-Time DETR):**

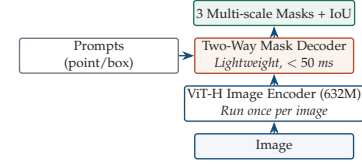
1. **Efficient Hybrid Encoder:** Decouples multi-scale feature interaction. Cross-scale feature fusion is handled purely by an efficient convolutional CSPRepLayer path; self-attention is applied *only* on the highest-level feature map (S_5), reducing encoder compute by 70%.
2. **Uncertainty-Minimal Query Selection:** Selects top- N queries based on classification confidence weighted by IoU scores, providing higher quality initial queries to the decoder. RT-DETR-R50 reaches 53.1 AP at 108 FPS on a T4 (TensorRT FP16), beating similarly sized YOLOs once their NMS time is counted; R101 gives 54.3 AP at 74 FPS.

HUNGARIAN MATCHING PITFALLS

- ▶ **Classification cost without probabilities:** In $\mathcal{L}_{\text{match}}$, using raw logits instead of probabilities $\hat{p}(c)$ creates scale imbalance against the bounded GIoU loss ($[-1, 1]$), causing matcher collapse.
- ▶ **Gradients through matcher:** The Hungarian algorithm permutation $\hat{\sigma}$ is non-differentiable and solved on CPU via linear sum assignment (`scipy.optimize.linear_sum_assignment`). Gradients flow only through the loss evaluated at the fixed indices $\hat{\sigma}$.

10 Image & Video Segmentation: U-Net to SAM 2

[U-Net · DeepLab ASPP · Mask R-CNN · Mask2Former · SAM · SAM 2]



The SAM Architecture. Splits heavy perception from interactive decoding.

- ▶ **Asymmetric design:** The heavy ViT image encoder produces a $64 \times 64 \times 256$ spatial embedding once. Interactive clicks/prompts run purely through the sub-50ms decoder in real time.
- ▶ **Ambiguity awareness:** Predicts 3 valid hierarchy masks (whole, part, subpart) to handle ambiguous single-point clicks.

SEGMENTATION PARADIGMS COMPARED

Task	Target Output	Canonical Benchmark
Semantic	Every pixel assigned class $c \in \{1, \dots, C\}$; no individual object separation.	ADE20K (mIoU), Cityscapes
Instance	Pixels grouped into discrete instance masks for "thing" categories (people, cars).	COCO Mask AP (AP_{mask})
Panoptic	Unified: Instance masks for countable "things" + semantic masks for amorphous "stuff" (sky, grass).	COCO Panoptic Quality ($PQ = SQ \times RQ$)

INTERVIEW Q&A | SEGMENTATION MECHANICS & MODERN FOUNDATION MODELS

Q39. Why are U-Net skip connections necessary for precise biomedical and boundary segmentation?

Successive pooling and strided convolutions downsample feature maps to capture high-level semantic context, destroying fine spatial details and precise pixel locations. **U-Net skip connections** concatenate shallow encoder features directly with decoder feature maps at the matching scale: Decoder receives both:

1. Low-frequency semantic abstractions from the bottleneck.
 2. High-frequency boundary and edge signals directly from the early encoder.
- Enables exact boundary localization without requiring dilated convolutions.

Q40. How does DeepLabv3+ use Atrous Spatial Pyramid Pooling (ASPP) to capture multi-scale context?

Chen et al. address the trade-off between receptive field and feature resolution using atrous/dilated convolutions with rate r : $K' = K + (K - 1)(r - 1)$. **ASPP Architecture:** Processes incoming feature map x in parallel through 5 branches:

1. One 1×1 convolution (preserves local details).
 2. Three 3×3 atrous convolutions with rates $r = [6, 12, 18]$ (samples distant context without reducing resolution).
 3. Image-level Global Average Pooling followed by 1×1 conv and bilinear upsampling (captures global image prior).
- All branches are concatenated along channels and projected through a 1×1 conv to produce multi-scale context representation.

Q41. How does Mask2Former unify semantic, instance, and panoptic segmentation?

Cheng et al. discard task-specific heads (RoIAlign, FCN) in favor of **Masked-Attention Transformers**:

1. Employs N object queries that predict binary mask logits $m_i \in [0, 1]^{H \times W}$ and class distribution p_i .
2. **Masked Attention:** Modulates decoder cross-attention to attend *only* within the foreground area predicted by the previous layer's

mask:

$$\mathcal{M}_{l-1}(x,y) = \begin{cases} 0 & \text{if } m_{l-1}(x,y) > 0.5 \\ -\infty & \text{otherwise} \end{cases}$$

Focuses attention on relevant localized object features, dramatically accelerating convergence and outperforming specialized models across all 3 tasks.

Q42. How does Mask R-CNN predict instance masks, and why a per-class sigmoid?

A third head runs on every positive RoI: RoIAlign to 14×14 , four 3×3 convs, a $2 \times$ deconv, then a 1×1 conv giving K masks of 28×28 (one per class). **Loss:** binary cross-entropy on the mask of the *ground-truth class only*, $\mathcal{L} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{box}} + \mathcal{L}_{\text{mask}}$.

Why decouple: a per-pixel softmax over classes makes masks compete; the paper reports about +5.5 mask AP for per-class sigmoids, because the box head already decides the class.

Inference: take the mask for the predicted class, resize the 28×28 map to the box, threshold at 0.5. The low 28^2 resolution is why boundaries look blobby (PointRend and Mask2Former fix this).

SEGMENTATION METRICS

- **mIoU vs Pixel Accuracy:** Pixel accuracy is easily distorted by background dominance (predicting background on 95% of pixels gives 95% accuracy but near-zero mIoU). Always optimize and report mean Intersection-over-Union: $\text{mIoU} = \frac{1}{C} \sum_c \frac{TP_c}{TP_c + FP_c + FN_c}$.
- **Dice Loss gradient near empty masks:** $\text{Dice} = \frac{2|X \cap Y| + \epsilon}{|X| + |Y| + \epsilon}$. If ϵ is too small ($< 10^{-7}$), division by tiny denominator when both prediction and ground truth are empty yields numerical instability.

11 Foundation Segmentation: SAM & SAM 2

[SAM ViT-H · prompt encoder · two-way decoder · ambiguity resolution · SAM 2 streaming video · memory bank]

SAM VS SAM 2 ARCHITECTURE COMPARISON

Component	SAM (Images, 2023)	SAM 2 (Images & Videos, 2024)
Image Backbone	ViT-H/L/B (MAE pretrained, 1024×1024 , windowed attn)	Hiera hierarchical backbone (multiscale, faster execution)
Prompt Types	Points (pos/neg), bounding boxes, dense masks	Points, boxes, masks across any video timestep
Decoder	Two-way cross-attention transformer (prompt $\leftrightarrow$ image)	Two-way transformer + Occlusion prediction head
Temporal State	None (single static frame)	Streaming Memory Bank (FIFO past frames + prompted frames)
Memory Attention	N/A	L-layer cross-attention attending to stored memory features
Dataset	SA-1B (11M images, 1.1B masks)	SA-V (51k videos, 600k masklets, $53\times$ more masks than prior)

INTERVIEW Q&A | SEGMENT ANYTHING (SAM) DEEP DIVE

Q43. How does the SAM architecture decouple the heavy image encoder from the real-time prompt decoder?

Decoupled Design:

1. **Image Encoder (Heavyweight):** Uses ViT-H (632M parameters) with input resolution 1024×1024 . To avoid quadratic self-attention cost across all $64 \times 64 = 4096$ patches, it uses interleaved local window attention with only 4 global attention blocks. Image encoding takes ~ 450 ms on an A100 GPU, but runs **exactly once** per image to output a spatial feature embedding $\mathbf{F} \in \mathbb{R}^{64 \times 64 \times 256}$.
2. **Prompt Encoder (Lightweight):**
 - **Sparse prompts (Points/Boxes):** Point (x,y) is represented by 2D Fourier positional encoding plus learned embeddings indicating point type (foreground vs background). Bounding boxes are encoded as two points (top-left and bottom-right).
 - **Dense prompts (Masks):** Downsampled via convolutional layers and element-wise added to the image embedding.
3. **Mask Decoder (Real-time):** A tiny two-way transformer (~ 4 M

parameters). Prompt tokens attend to image embeddings, and image embeddings attend to prompt tokens. Decodes in < 50 ms on modern GPUs, enabling instant, fluid interactive browser annotation.

Q44. How does SAM solve the mathematical ambiguity of a single point prompt without collapsing training gradients?

The Ambiguity Problem: A single click on an person's shirt could mean (1) the shirt button, (2) the shirt itself, or (3) the entire person. Training a standard network with binary cross-entropy on ground-truth masks produces blurry, averaged probabilities that attempt to satisfy all three meanings simultaneously.

SAM Ambiguity Resolution:

- The mask decoder always outputs **3 distinct mask tokens** simultaneously, representing subpart, part, and whole object.
- During training, SAM calculates the loss against ground-truth for all 3 masks, but backpropagates gradients **only through the single mask with the lowest loss** (minimum error backpropagation).
- In addition, an auxiliary MLP head predicts the IoU score for each mask.
- At inference, if ambiguous, SAM displays the 3 candidate masks or selects the one with the highest predicted IoU score.

INTERVIEW Q&A | SAM 2: VIDEO OBJECT SEGMENTATION & MEMORY DYNAMICS

Q45. How does SAM 2 extend promptable segmentation to continuous video streams in real time?

SAM 2 introduces a causal streaming architecture that processes video frame-by-frame:

- **Memory Attention Block:** When a new frame t arrives, its Hiera visual features attend to previous spatial-temporal representations stored in the **Memory Bank** via stacked cross-attention layers.
- **Streaming Memory Bank:** Stores two distinct types of memories:
 1. **Temporal FIFO:** A sliding window of features from the most recent N frames (capturing immediate motion and appearance transitions).
 2. **Prompted Anchors:** Features from all frames where the user provided explicit interactive prompts (preventing catastrophic forgetting and drift over long videos).
- **Memory Encoder:** Once a mask is predicted for frame t , a lightweight convolutional memory encoder downsamples the mask, fuses it with image features, and writes the compact embedding into the memory bank.
- **Occlusion Head:** Predicts a probability $p_{\text{occ}} \in [0,1]$ indicating whether the target object is currently visible. If occluded, the memory bank suppresses visual updates to avoid contaminating tracking with background clutter.

Q46. Why did SAM 2 replace the ViT backbone with Hiera, and what is the latency advantage?

ViT-H limitations: Standard ViT operates at a single fixed scale with isotropic resolution, requiring costly windowed attention and large FLOP budgets (~ 632 M parameters, ~ 50 ms per frame even after caching).

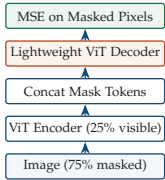
Hiera Backbone: A hierarchical vision transformer that produces multi-scale feature maps ($4\times, 8\times, 16\times, 32\times$) with simple masked autoencoder pretraining. It removes complex operations (no shifted windows, no relative position bias tables, no multi-path branches) and relies purely on simple spatial pooling. Result: on images SAM 2 is **about $6\times$ faster** than SAM at higher accuracy, and the Hiera-B+ model tracks video at ~ 44 FPS on an A100.

SAM / SAM 2 PRACTICAL TRAPS

- **Semantic Class Agnosticism:** SAM predicts *masks*, not semantic *categories*. It outputs "where" an object is, but has zero awareness of "what" it is. For semantic segmentation, SAM must be paired with CLIP/Grounding-DINO.
- **Memory Bank GPU Leak in SAM 2:** In long videos (> 1000 frames), an unbounded memory bank exhausts GPU VRAM. SAM 2 keeps only the 6 most recent unprompted frames plus prompted frames by default; when running your own streaming loop, cap both and evict FIFO.

12 Self-Supervised Vision: Contrastive, MAE & DINOv2

[SimCLR · MoCo v1-v3 · BYOL · MAE · DINO · DINOv2 · KoLeo]



The MAE Paradigm. Exploits heavy 2D pixel redundancy.

- **Asymmetric architecture:** Only the 25% unmasked patches pass through the heavy encoder. The shallow decoder reconstructs normalized pixels.
- **Speedup:** 4× compute reduction and memory savings during pretraining compared to standard ViT.

SSL VISION PARADIGMS COMPARISON

Method	Core Loss / Objective	Collapse Prevention Mechanism
SimCLR	InfoNCE with negative pairs across batch	Massive batch size ($B = 4096$) + non-linear projection head.
MoCo v1-v3	InfoNCE over persistent memory queue	Momentum-updated key encoder ($\theta_k \leftarrow m\theta_k + (1-m)\theta_q$).
BYOL / SimSiam	MSE between online predictor and target	Stop-gradient ($\text{sg}[\cdot]$) on target branch + asymmetric predictor MLP.
MAE	Pixel MSE on masked patches	Reconstruction task (no negative pairs; cannot collapse to constant).
DINO / DINOv2	Cross-entropy student vs momentum teacher	Logit centering (prevents 1-class mode) + sharpening (prevents uniform mode).

INTERVIEW Q&A | SELF-SUPERVISED MECHANICS & REPRESENTATION GEOMETRY

Q47. Derive why InfoNCE minimizes the lower bound on Mutual Information $I(X; Y)$.

Let (x, y) be positive pairs and $\{y_k^-\}_{k=1}^K$ be K negative samples drawn from marginal $p(y)$. The InfoNCE loss is:

$$\mathcal{L}_{\text{InfoNCE}} = -\mathbb{E} \left[\log \frac{e^{f(x, y)}}{\sum_{k=1}^K e^{f(x, y_k^-)}} \right]$$

By Jensen's inequality and properties of Bayes' rule:

$$\begin{aligned} \mathcal{L}_{\text{InfoNCE}} &\geq \mathbb{E} \left[\log \left(1 + (K-1) \frac{p(y)}{p(y|x)} \right) \right] \\ &\approx \log(K) - I(X; Y) \implies I(X; Y) \geq \log(K) - \mathcal{L}_{\text{InfoNCE}} \end{aligned}$$

Minimizing InfoNCE maximizes the mutual information lower bound between augmented views. Larger batch/queue size K directly tightens the bound $\log(K)$.

Q48. Why doesn't BYOL collapse even though it has no negative pairs?

Online network $f_\theta \rightarrow g_\theta \rightarrow q_\theta$ (predictor) regresses the target projection of the other view: $\mathcal{L} = \|\bar{q}_\theta(z) - \bar{z}'_\xi\|_2^2$ (L2-normalized), with target weights $\xi \leftarrow m\xi + (1-m)\theta$, $m: 0.996 \rightarrow 1$. A constant output is a minimum of this loss, yet training does not reach it, because:

1. **Stop-gradient on the target:** the target is not optimized toward the online output, so the two branches cannot jointly shrink to a point.
2. **Predictor:** SimSiam shows that removing q_θ collapses training immediately; a near-optimal predictor makes the online branch track the target's conditional expectation, which keeps variance.
3. **EMA target:** slowly moving targets stabilize the dynamics (SimSiam shows EMA is helpful but not required).

Interview line: *asymmetry (predictor + stop-grad) prevents collapse; negatives are not required.*

Q49. How does DINO produce emergent semantic segmentation without annotations?

Caron et al. train a ViT student to match a momentum teacher over multi-crop views. Because the objective maximizes agreement between global crops (full image) and local crops (sub-regions), and because self-attention in transformers connects all spatial tokens without local receptive field restrictions: The [CLS] and patch tokens learn to cluster around distinct foreground objects to maximize mutual information across varied scales. Attention maps from the

last layer of DINO explicitly outline foreground objects without any segmentation labels or bounding boxes.

Q50. What makes DINOv2 the gold-standard frozen vision feature extractor?

Oquab et al. scale self-distillation to 1B parameters (ViT-g):

1. **Curated LVD-142M Data Pipeline:** De-duplicates, filters, and balances 142M images using nearest-neighbor embedding retrieval against ImageNet-22k.
2. **Joint iBOT Objective:** Applies token-level masked image modeling simultaneously with view-level DINO self-distillation.
3. **KoLeo Regularizer:** Adds Kozachenko-Leonenko differential entropy estimator minimizing $-\sum_i \log \min_{j \neq i} \|z_i - z_j\|_2$, forcing visual feature embeddings to spread uniformly over the unit hypersphere. Freezing DINOv2 features outperforms weakly supervised models on zero-shot classification, monocular depth estimation, and dense feature matching.

SSL PRETRAINING TRAPS

- **Discarding the projection head:** In SimCLR, MoCo, and BYOL, downstream evaluation must use the encoder representation $h = f(x)$, **not** the projection head output $z = g(h)$. The projection head discards information (e.g., color, orientation) irrelevant to the contrastive objective that is vital for downstream tasks.
- **Comparing SSL methods on the wrong protocol:** MAE looks weak on linear probe and k-NN but strong after fine-tuning; DINO/DINOv2 are the reverse. Always report linear probe, k-NN and fine-tune (and say whether the backbone is frozen) before claiming one method wins.

13 SSL Deep Dive: MAE, DINO & DINOv2

[MAE asymmetric encoder · 75% masking · DINO centering & sharpening · KoLeo regularizer · Register tokens]

MAE VS DINOv2 PARADIGM COMPARISON

Property	Masked Autoencoders (MAE)	Self-Distillation (DINOv2)
Objective	Reconstruct masked pixels (L_2 MSE)	Match teacher probability distribution (Cross-Entropy)
Supervision	Generative reconstruction	Joint embedding self-distillation
Masking Views	75% random masking of patches	Multi-crop (2 global views + 8 local views)
Asymmetry	Encoder sees only 25% visible patches	Student sees local+global; Teacher sees global only
Features	Strong fine-tuning backbone; poor k-NN	State-of-the-art frozen features; > 86% linear probe
Regularizer	Normalized patch pixel targets	KoLeo differential entropy + Register tokens

INTERVIEW Q&A | MASKED AUTOENCODERS (MAE) MECHANICS

Q51. Why does MAE mask 75% of image patches, whereas BERT masks only 15% of text tokens?

Information Density: Language is highly information-dense, discrete, and semantic; masking 15% of words in a sentence already leaves subtle, challenging cloze puzzles. In contrast, images are continuous, spatially redundant signals: adjacent pixels are heavily correlated.

- At 15% image masking, a network can reconstruct missing pixels through trivial local spatial interpolation (blurring/edge continuation) without understanding high-level scene composition.
- At **75% masking**, only 1 in 4 patches is visible. The model cannot interpolate locally; it is forced to learn holistic scene grammar (e.g., inferring that an eye requires a nose, mouth, and face).

Q52. How does MAE achieve a 4x throughput acceleration during pretraining?

Asymmetric Architecture:

1. **Heavy Encoder:** Operates **strictly on visible patches**. For 196 total patches, 75% masking leaves only $N_{\text{vis}} = 49$ tokens. Since ViT self-attention scales quadratically, attention FLOPs drop by $\left(\frac{49}{196}\right)^2 = \left(\frac{1}{4}\right)^2 = \frac{1}{16}$!
2. **Lightweight Decoder:** Only after the heavy encoder has processed visible tokens are learned, shared [mask] tokens inserted at the

masked positions to form the complete 196-token sequence. The decoder is shallow (~ 8 layers vs 24 in encoder), incurring minimal compute.

3. **Loss Computation:** MSE is computed **only on masked patches**, further saving loss evaluation overhead.

INTERVIEW Q&A | DINO & DINOv2 SELF-DISTILLATION DYNAMICS

Q53. Derive how DINO avoids representation collapse without negative pairs via Centering and Sharpening.

In self-distillation, student g_s predicts teacher g_t output. Two failure modes (collapse) threaten training:

1. **Dirac collapse:** Output distribution collapses to a single constant coordinate for all inputs.
2. **Uniform collapse:** Output distribution collapses to a uniform distribution with maximum entropy.

DINO parameterizes probability outputs via temperatures τ_s, τ_t and center vector c :

$$P_s(x)^{(i)} = \frac{\exp(g_s(x)^{(i)} / \tau_s)}{\sum_k \exp(g_s(x)^{(k)} / \tau_s)}$$

$$P_t(x)^{(i)} = \frac{\exp((g_t(x)^{(i)} - c_i) / \tau_t)}{\sum_k \exp((g_t(x)^{(k)} - c_k) / \tau_t)}$$

- **Centering:** $c \leftarrow m \cdot c + (1 - m) \frac{1}{B} \sum_b g_t(x_b)$. Subtracting the center vector pulls the mean activation toward zero, preventing any single dimension from dominating (averts Dirac collapse). However, excessive centering pushes probabilities toward uniform distribution.
- **Sharpening:** Setting teacher temperature $\tau_t < \tau_s$ ($\tau_t \approx 0.04 - 0.07$ vs $\tau_s \approx 0.1$) sharpens the distribution, counteracting the uniform dispersion.
- **Equilibrium:** Centering and sharpening act as opposing forces: centering pushes toward uniform entropy, sharpening pushes toward peaked entropy. Balanced together, they guarantee stable non-collapsed representations.

Q54. What is the KoLeo regularizer in DINOv2, and how do Register Tokens eliminate feature map artifacts?

KoLeo (Kozachenko-Leonenko) Regularizer: Computes an empirical estimate of differential entropy across normalized feature vectors z_i :

$$\mathcal{L}_{\text{KoLeo}} = -\frac{1}{n} \sum_{i=1}^n \ln \left(\min_{j \neq i} \|z_i - z_j\|_2 \right)$$

Minimizing $\mathcal{L}_{\text{KoLeo}}$ maximizes the distance between every feature vector and its nearest neighbor, forcing representations to distribute uniformly across the unit hypersphere and preventing feature clustering.

Register Tokens (Darcet et al., 2023): Standard ViTs trained with DINO/DINOv2 exhibit severe high-norm spike artifacts in flat, uniform background regions (5x higher activation norms than foreground objects).

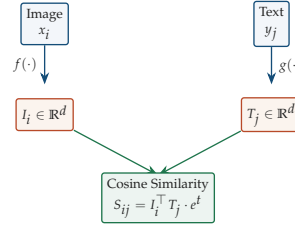
- **Cause:** Because ViT lacks dedicated scratchpad memory, self-attention recycles uninformative background tokens to store global contextual information.
- **Solution:** Append 4-8 learnable [register] tokens to the input patch sequence. The model automatically routes global context through registers, completely eliminating background visual artifacts and producing razor-sharp feature maps.

SSL PRETRAINING TRAPS

- **Patch Pixel Normalization in MAE:** Omitting per-patch mean/variance normalization ($\frac{x-\mu}{\sigma}$) when computing MAE reconstruction loss costs about 3 points of linear-probe accuracy (68.0 $\rightarrow$ 64.8 for ViT-L), as the model wastes capacity fitting global illumination variations rather than local structural geometry.
- **EMA Teacher Update Rate:** The teacher EMA momentum must start high ($\lambda = 0.996$) and cosine-anneal to 1.0 at the end of training. Setting λ too low makes the teacher track noisy student weights, destabilizing training.

14 Vision-Language Foundations: CLIP to SigLIP

[CLIP · dual encoder · InfoNCE · SigLIP sigmoid loss · zero-shot transfer · all-gather]



The Dual-Encoder Contrastive Setup. Embeds images and text into a shared normalized latent space $\mathbb{R}^d$ ($d = 512$ or 768).

► **Zero-Shot Transfer:** Candidate class names embedded with prompt template $\rightarrow$ cosine similarity $\rightarrow$ argmax class.

► **Temperature:** learned log-scale t , initialized at $\ln(1/0.07)$ and clamped to $\leq \ln 100 \approx 4.605$ to prevent training instability.

LOSS FORMULATIONS: CLIP VS SIGLIP

- **CLIP (Symmetric InfoNCE):** Computes cross-entropy over batch B :

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{2B} \sum_{i=1}^B \left(\log \frac{e^{S_{ii}}}{\sum_{j=1}^B e^{S_{ij}}} + \log \frac{e^{S_{ii}}}{\sum_{j=1}^B e^{S_{ji}}} \right)$$

Requires an all-gather of every embedding ($\mathcal{O}(Bd)$ traffic) and a full $B \times B$ logit matrix per GPU slice to form each softmax denominator; that memory bounds the batch size.

- **SigLIP (Sigmoid Loss):** Formulates language-image alignment as pairwise binary classification:

$$\mathcal{L}_{\text{SigLIP}} = -\frac{1}{B} \sum_{i=1}^B \sum_{j=1}^B \log \sigma \left(y_{ij} (I_i^T T_j \cdot e^t + b) \right)$$

where $y_{ij} = +1$ if $i = j$, else -1 ; init $t = \ln 10$, $b = -10$ (most pairs are negatives). No global normalization, so the loss can be computed chunk by chunk.

INTERVIEW Q&A | CONTRASTIVE MULTIMODAL MECHANICS & COMMUNICATION SCALING

Q55. Derive the gradient of the InfoNCE loss with respect to similarity logits S_{ij} .

For a single sample i , the image-to-text loss is $\mathcal{L}_i = -\log p_{ii}$ where $p_{ij} = \frac{e^{S_{ij}}}{\sum_k e^{S_{ik}}}$. Differentiating with respect to logit S_{ij} :

$$\frac{\partial \mathcal{L}_i}{\partial S_{ij}} = p_{ij} - y_{ij} \quad \text{where } y_{ij} = \mathbf{1}_{\{i=j\}}$$

- For the positive target ($j = i$): $\frac{\partial \mathcal{L}_i}{\partial S_{ii}} = p_{ii} - 1 < 0$ (pulls positive similarity upward).
- For negative targets ($j \neq i$): $\frac{\partial \mathcal{L}_i}{\partial S_{ij}} = p_{ij} > 0$ (pushes negative similarity downward proportional to its current probability mass). Softmax creates competition: pushing up the positive logit suppresses all negatives simultaneously, but requires summing over all negatives in the denominator.

Q56. Why does SigLIP scale to $10\times$ larger batch sizes than CLIP in distributed clusters?

In standard CLIP, computing $\sum_{j=1}^B e^{S_{ij}}$ requires every GPU to know the embeddings of all B images and texts across the entire cluster. In distributed data parallel (DDP) across N GPUs, this necessitates an **All-Gather** collective communication operation of all embedding tensors ($2 \times B \times d$). At $B = 65,536$ the all-gather itself is modest (Sec. 29: ~ 100 MB), but each GPU must hold its $b \times B$ slice of logits plus activations for the backward pass. **SigLIP Solution:** Binary classification evaluates independent pairs (i, j) without a global partition function. Each GPU computes the loss on its local block, then passes its text embeddings to a neighbour in a ring (D steps for D devices), so only $b \times b$ logits exist at once. The paper trains up to $B = 1\text{M}$ but finds gains saturate around 32k, and SigLIP beats CLIP most clearly at small batches.

Q57. Why is zero-shot CLIP significantly more robust to distribution shift than supervised ImageNet models?

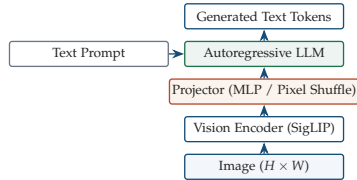
Supervised ResNet models learn shortcut features (high-frequency background textures, camera color balances, specific noise profiles) correlated with ImageNet labels. When tested on shifted distributions (ImageNet-A adversarial examples, ImageNet-R art/sketches, ImageNet-C corruptions), supervised accuracy drops precipitously ($\sim 40\%$ drop). CLIP is pretrained on 400M diverse internet image-caption pairs using natural language supervision: It learns invariant semantic concepts (e.g., "whiskers", "four legs", "stripes") rather than single-dataset statistical artifacts. On distribution-shifted benchmarks, CLIP retains up to 20–30% higher accuracy than supervised equivalents with identical ImageNet-1K top-1 accuracy.

DUAL ENCODER PITFALLS

- **L2 Normalization before dot product:** Both visual embeddings I and text embeddings T must be L2-normalized ($\|I\|_2 = 1, \|T\|_2 = 1$) before the dot product and temperature scaling. Omitting normalization breaks the cosine geometry, allowing outlier vector lengths to dominate the loss.
- **Prompt sensitivity:** Evaluating single-word class names ("dog") yields 5 – 10% lower zero-shot accuracy than prompt ensembles (averaging embeddings of 80 templates: "a photo of a label", "a close-up of the label").

15 Multimodal LLMs: Architecture, AnyRes & Alignment

[LLaVA · Projectors · AnyRes dynamic resolution · 2D RoPE · Qwen2-VL · Hallucination & POPE]



The Modern VLM Pipeline. Integrates visual tokens directly into the LLM's sequence context.

- **Token budget:** A 336×336 image with patch 14 produces $(336/14)^2 = 576$ visual tokens.
- **Pixel Shuffle:** Merging 2×2 adjacent patch tokens compresses sequence length by $4\times$ ($576 \rightarrow 144$ tokens), preserving LLM context budget.

VISUAL CONNECTORS / PROJECTORS

Connector	Mechanism	Trade-off
Linear / 2-Layer MLP (LLaVA)	$H_v = W_2 \cdot \text{GELU}(W_1 \cdot Z_v)$	Preserves spatial topology 1:1; high token count in LLM context.
Perceiver / Q-Former (BLIP-2)	K learned query tokens cross-attend to visual features	Compresses visual sequence to fixed size (e.g. 64); loses fine-grained dense OCR details.
Pixel Shuffle / Merge (InternVL, Qwen2)	Reshapes 2×2 spatial tokens into channels: $(H/2, W/2, 4C) \rightarrow (H/2, W/2, C_{\text{LLM}})$	$4\times$ token reduction while maintaining 2D spatial relative geometry.

INTERVIEW Q&A | HIGH-RESOLUTION & ALIGNMENT

Q58. How does the AnyRes (grid slicing) strategy handle high-resolution documents and charts?

Standard ViTs downsample images to fixed small resolutions (e.g., 336×336), rendering small text and chart labels unreadable. **AnyRes Strategy (LLaVA-NeXT):**

1. Determines an optimal grid of tiles (e.g., $1 \times 2, 2 \times 2, 1 \times 3$) based on image aspect ratio.
 2. Crops image into N standard 336×336 high-resolution patches, plus one downsampled global image for macro context.
 3. Feeds all $N + 1$ tiles through the ViT encoder independently.
 4. Inserts special separator tokens (`<newline>`) between rows to maintain 2D layout semantics when flattened into the 1D LLM token sequence.
- Increases effective resolution up to $4\times$ while reusing pretrained small-resolution ViT weights.

Q59. How does Qwen2-VL implement native dynamic resolution with 2D Rotary Position Embeddings?

Rather than slicing images into fixed tiles, Qwen2-VL uses **Naive Dynamic Resolution**:

- **Dynamic Patchification:** An image is resized only to the nearest multiple of 28, split into $\frac{H}{14} \times \frac{W}{14}$ patches, then adjacent 2×2 patches are merged by an MLP, so a 224^2 image costs 64 LLM tokens.
- **2D RoPE (Rotary Position Embeddings):** Query and key vectors at spatial coordinate (x, y) are rotated by two independent 1D frequency angles:

$$\mathbf{R}_{2D}(x, y) = \text{diag}(\mathbf{R}(x), \mathbf{R}(y))$$

Inner products $\langle \mathbf{q}_{(x_1, y_1)}, \mathbf{k}_{(x_2, y_2)} \rangle$ depend strictly on relative spatial distance $(\Delta x, \Delta y) = (x_1 - x_2, y_1 - y_2)$.

- **M-RoPE:** in the LLM, rotary dimensions are split into temporal, height and width components (t, h, w) ; text tokens use equal ids for

all three, so text, images and video share one position scheme. Eliminates tile boundaries and aspect ratio distortion completely.

Q60. Walk through the LLaVA training recipe. What is frozen at each stage?

1. **Stage 1, feature alignment** (LLaVA-1.5: 558K image-caption pairs, 1 epoch): vision encoder *and* LLM frozen; only the MLP projector trains, learning to map CLIP/SigLIP patches into the LLM's word-embedding space. Cheap: a few GPU-hours.
2. **Stage 2, visual instruction tuning** (665K mixed conversations: GPT-generated QA, VQAv2, GQA, OCR, region data, plus ShareGPT text): projector + LLM train (full or LoRA); vision encoder usually stays frozen (later models unfreeze it at a low LR).
3. **Stage 3 (modern additions):** high-resolution/AnyRes data, then preference tuning (DPO/RLHF-V) against hallucination. Loss is next-token cross-entropy on assistant tokens only. Text-only data stays in the mix so the LLM does not forget language skills.

MULTIMODAL TRAINING PITFALLS

- **Unfreezing vision encoder too early:** Unfreezing the vision encoder during Stage 1 (feature alignment) with random MLP projector weights corrupts the pretrained visual representations. Stage 1 must train *only* the projector.
- **Loss masking:** Next-token cross-entropy loss must be computed *only* on the assistant's generated text tokens. Labels for user prompt text and visual tokens must be masked to `-100` (`ignore_index`).

16 VLM Evaluation & Visual Hallucination

[POPE benchmark · affirmative bias · Visual Contrastive Decoding (VCD) · MMMU · DocVQA · OCR]

KEY VISION-LANGUAGE BENCHMARKS TO MEMORIZE

Benchmark	Evaluation Focus	Core Challenge / Metric
POPE	Object hallucination	Random, Popular, Adversarial; F1 & Yes-ratio
MMMU	College reasoning	30 subjects, diagrams, physics, schematics
MathVista	Visual math reasoning	Geometry proofs, coordinate plots, arithmetic
DocVQA	Document comprehension	Fine-grained OCR, invoice tables, font parsing
ChartQA	Data visualization	Numerical reasoning over bar charts, scatter plots
MME	Perception & cognition	Yes/No probing with circular evaluation

INTERVIEW Q&A | THE POPE BENCHMARK & OBJECT HALLUCINATION MECHANICS

Q61. How does POPE (Polling-based Object Probing Evaluation) expose visual hallucination, and what are its 3 splits?

Standard open-ended captioning metrics (CIDEr, BLEU) fail to detect hallucinations: a model can produce a beautifully fluent caption with 90% true text and casually invent a dog or chair.

POPE Probing Format: Converts object existence into binary Yes/No queries: "Is there a `<object>` in the image?"

- **Random Split:** Queries non-existing objects selected purely at random from the dataset lexicon. (Tests baseline vocabulary discrimination).
 - **Popular Split:** Queries non-existing objects chosen from the most frequent objects in the dataset (e.g., person, car, chair). Exposes frequency bias where models hallucinate popular classes.
 - **Adversarial Split:** Queries non-existing objects that frequently **co-occur** with objects actually present in the image. (e.g., an image contains a laptop; POPE queries whether a mouse is present). Tests whether the model looks at visual evidence or blindly relies on statistical text priors.
- Yes-Ratio Diagnostic:** A model reporting 80% accuracy might have a 95% "Yes" response rate; checking Precision, Recall, F1, and Yes-ratio prevents false performance inflation.

Q62. Why do VLMs hallucinate, and how does Visual Contrastive Decoding (VCD) mitigate it without training?

Root Causes of Hallucination:

1. **Language Prior Over-Reliance:** The LLM backbone's strong pre-training causes language statistical priors to overpower visual tokens (e.g., predicting "spoon" after "fork").
2. **Attention Sinks & Visual Degradation:** In long autoregressive

generation, attention to visual tokens decays exponentially; the model defaults to pure text continuation.

Visual Contrastive Decoding (VCD, Leng et al., 2024):



- Creates a corrupted image v_{corrupt} by adding Gaussian noise or blurring to destroy fine-grained visual details while keeping coarse text prompt q identical.
- Evaluates both original logits $p(y_t|v, q, y_{<t})$ and corrupted logits $p(y_t|v_{\text{corrupt}}, q, y_{<t})$.
- Contrasts the two distributions at each decoding step:

$$\text{logit}^*(y_t) = (1 + \beta) \text{logit}(y_t|v) - \beta \text{logit}(y_t|v_{\text{corrupt}}) \quad (\beta \approx 0.5 - 1.0)$$

- Tokens driven purely by language priors have equal probability under both views and cancel out; tokens strongly grounded in visual evidence remain elevated.

INTERVIEW Q&A | HIGH-RESOLUTION DOCUMENT UNDERSTANDING & MMMU

Q63. What makes document understanding (DocVQA) fundamentally harder than natural image VLM tasks?

- Aspect Ratio & Scale:** Documents are typically tall portrait orientations (8.5×11 inches, $> 2000 \times 3000$ pixels). Resizing to square 336×336 or 448×448 shrinks 8pt fonts to sub-pixel noise, causing severe OCR failure.
- Reading Order & Spatial Layout:** Text flows across multi-column tables, bounding boxes, and headers. Raster visual encoders lack explicit layout priors and scramble reading order without specialized 2D positional embeddings.
- Resolution Solution:** Modern models (Qwen2-VL, InternVL-2) use dynamic patch slicing (AnyRes) or native resolution processing, dividing high-res documents into 10 – 20 localized sub-patches and preserving native 300 DPI text fidelity.

Q64. Why is MMMU considered the premier college-level evaluation for multimodal foundation models?

Unlike perception benchmarks testing whether a model sees a cat, MMMU evaluates expert reasoning across 30 college-level disciplines (Quantum Physics, Organic Chemistry, Structural Engineering, Medicine). Questions involve multi-panel plots, circuit schematics, histological slides, and complex mathematical formulas, requiring joint perceptual parsing, spatial geometric projection, and multi-step domain reasoning.

VLM EVALUATION PITFALLS

- Option Order Bias:** Multiple-choice benchmarks (e.g. MMBench) suffer from severe position bias (e.g., models prefer option "A" 60% of the time). Always run **circular evaluation** (evaluating all cyclic permutations $A \rightarrow B \rightarrow C \rightarrow D$ and requiring correct answers on all permutations to grant a point).
- Data Contamination:** Many popular vision-language benchmarks (DocVQA, ScienceQA) have had their images scraped into web-scale pretraining datasets; always verify train-test split hashes.

17 Generative Diffusion: DDPM, DDIM & Guidance

[DDPM · score SDE/ODE · simplified loss · DDIM ODE sampling · EDM · CFG]



The Diffusion Duality. Gradually corrupts data with Gaussian noise in the forward process; trains a neural network $\epsilon_\theta(x_t, t)$ to reverse the trajectory step-by-step.

- One-step noising:** Can jump directly to any timestep t without simulating intermediate steps: $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$, where $\epsilon \sim \mathcal{N}(0, \mathbf{I})$.

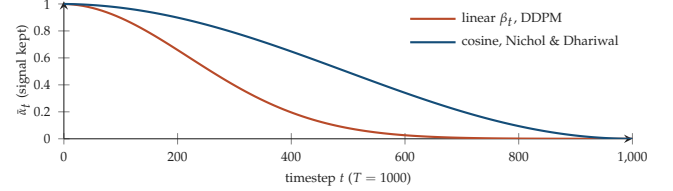
MATHEMATICAL FOUNDATIONS OF DDPM

- Noise schedule:** $\beta_1 < \dots < \beta_T$; $\alpha_t = 1 - \beta_t$; $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$.
- Training Objective ($\mathcal{L}_{\text{simple}}$):** Predicts added noise ϵ via MSE:

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t \sim \mathcal{U}(1, T), x_0, \epsilon \sim \mathcal{N}(0, \mathbf{I})} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

- Reverse Step:** Reconstructs x_{t-1} from x_t :

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z \quad (z \sim \mathcal{N}(0, \mathbf{I}))$$



The linear schedule destroys almost all signal by $t \approx 600$, wasting the last 40% of steps on near-pure noise; the cosine schedule degrades smoothly.

INTERVIEW Q&A | SAMPLING ACCELERATION, SDE THEORY & CLASSIFIER-FREE GUIDANCE

Q65. Derive the connection between DDPM and Continuous Score-Based SDEs.

Song et al. unify DDPM as the discretization of an Itô Stochastic Differential Equation:

- Forward SDE:** $dx = f(x, t)dt + g(t)d\mathbf{w}$, where drift $f(x, t) = -\frac{1}{2}\beta(t)x$ and diffusion $g(t) = \sqrt{\beta(t)}$.

- Reverse-Time SDE:**

$$dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)] dt + g(t)d\bar{\mathbf{w}}$$

where $\nabla_x \log p_t(x)$ is the **score function**. Tweedie's formula reveals that the score is directly proportional to predicted noise:

$$\nabla_x \log p_t(x) = -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}$$

- Probability Flow ODE:** Removing the Brownian motion term $d\bar{\mathbf{w}}$ yields a deterministic ODE with identical marginal distributions:

$$dx = \left[f(x, t) - \frac{1}{2}g(t)^2 \nabla_x \log p_t(x) \right] dt$$

Solvable using high-order numerical solvers (Runge-Kutta, DPM-Solver) in 15 – 20 steps.

INTERVIEW Q&A | EDM FRAMEWORK & CFG DYNAMICS

Q66. Why does EDM (Karras et al., 2022) replace discrete schedules with continuous noise σ ?

Discrete β -schedules tie model weights to specific discrete step counts ($T = 1000$). **EDM Formulation:** Parametrizes corruption continuously via noise standard deviation σ : $\mathbf{x} = \mathbf{y} + \mathbf{n}$, $\mathbf{n} \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$. Employs network preconditioning $D_\theta(\mathbf{x}; \sigma) = c_{\text{skip}}(\sigma)\mathbf{x} + c_{\text{out}}(\sigma)F_\theta(c_{\text{in}}(\sigma)\mathbf{x}; c_{\text{noise}}(\sigma))$ such that effective training loss variance remains constant across all noise levels:

$$\mathcal{L}_{\text{EDM}} = \mathbb{E}_{\mathbf{y}, \sigma, \mathbf{n}} [\lambda(\sigma) \|D_\theta(\mathbf{y} + \mathbf{n}; \sigma) - \mathbf{y}\|_2^2]$$

Samples $\ln(\sigma) \sim \mathcal{N}(P_{\text{mean}}, P_{\text{std}}^2)$, focusing model capacity on the mid-range noise levels that determine visual structure.

Q67. How does Classifier-Free Guidance (CFG) work, and why does high guidance cause image burn?

Mechanism: In a text-to-image model, conditioning prompt c is dropped with probability $p_{\text{uncond}} \approx 0.1 - 0.2$ during training (replaced with null embedding $\emptyset$). At inference, both conditional and unconditional predictions are evaluated, and the score is extrapolated by guidance scale $s \geq 1.0$:

$$\tilde{\epsilon}_\theta(x_t, c) = \epsilon_\theta(x_t, \emptyset) + s \cdot (\epsilon_\theta(x_t, c) - \epsilon_\theta(x_t, \emptyset))$$

The trade-off:

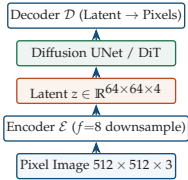
- $s = 1.0$: No guidance; samples follow true unconditional data distribution with high diversity but weak text alignment.
- $s \in [5, 7.5]$: Optimal zone; sharpens semantics and adherence to prompt.

- ▶ $s > 10$: Extrapolation forces latent values into extreme tails outside the training distribution, causing clipped latents, unnatural contrast, saturated colors, and "burn" artifacts.
- DIFFUSION NUMERICAL TRAPS

 - ▶ **Timestep Embedding**: Timestep t must be mapped via sinusoidal positional embeddings followed by a 2-layer MLP before injection into ResNet/Attention blocks; feeding raw scalar integer t prevents scale-invariant time conditioning.
 - ▶ **Variance Preserving vs Exploding**: DDPM uses Variance Preserving (VP) formulation ($\tilde{\alpha}_t + (1 - \tilde{\alpha}_t) = 1$). EDM uses Variance Exploding (VE) where noise variance grows without bound: $x_t = x_0 + \sigma_t \epsilon$.

18 Latent Diffusion & Diffusion Transformers (DiT)

[LDM · Stable Diffusion · VAE compression · DiT · adaLN-Zero · SDXL]



Perceptual vs Semantic Compression. Pixel space diffusion wastes $> 90\%$ of capacity on imperceptible high-frequency pixel noise.

- ▶ $f = 8$ **Compression**: $512 \times 512 \times 3 = 786,432$ values compressed to $64 \times 64 \times 4 = 16,384$ values ($48\times$ memory reduction).
- ▶ **Cross-modal injection**: Text conditioning injected into spatial latents via cross-attention.

LATENT DIFFUSION MODELS COMPARED

Model	Params	Text Encoders	Conditioning / Backbone
SD 1.5	860M	CLIP ViT-L/14	UNet 512 ² , Cross-attention
SD 2.1	865M	OpenCLIP-ViT-H	UNet 768 ² , v -pred, cross-attn
SDXL	2.6B	CLIP-L OpenCLIP-G	+ UNet 1024 ² , Dual text, crop coords
DiT-XL/2	675M	Class / CLIP tokens	Transformer 256 ² , adaLN-Zero

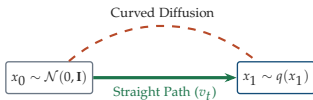
INTERVIEW Q&A | ARCHITECTURE SHIFTS: UNET TO DiT

- Q68. Why did modern generative vision abandon convolutional UNets for Diffusion Transformers (DiT)?**
1. **Scaling Laws**: Convolutional UNets plateau in generation quality as parameter counts scale past 2B due to fixed receptive fields and localized inductive biases. Transformers scale monotonically with compute and FLOPs without saturation. 2. **Flexible sequence handling**: UNets require rigid spatial grid tensors. DiT patchifies latents into 1D token sequences, making it trivial to process arbitrary aspect ratios, variable image resolutions, and temporal video frames in a single unified architecture. 3. **Hardware efficiency**: Standard dense matrix multiplications in Transformer self-attention achieve $> 50\%$ MFU on Tensor Cores (especially with FlashAttention-2), whereas multi-scale UNet up/down convolutions suffer from high memory bandwidth overhead.
- Q69. How does adaLN-Zero work in DiT, and why is zero-initialization necessary?**
- Rather than injecting timestep and class embeddings via cross-attention, DiT uses **Adaptive Layer Normalization (adaLN)**: A projection MLP maps the sum of timestep and condition embeddings ($t + c$) into scale, shift, and gate parameters ($\gamma_1, \beta_1, \alpha_1, \gamma_2, \beta_2, \alpha_2$):
- $$h' = h + \alpha_1 \cdot \text{Attention}((1 + \gamma_1) \odot \text{LN}(h) + \beta_1)$$
- $$h_{\text{out}} = h' + \alpha_2 \cdot \text{MLP}((1 + \gamma_2) \odot \text{LN}(h') + \beta_2)$$
- Zero-Initialization**: The gating factors α_1, α_2 are initialized to **zero**. At initialization, every DiT block acts as an exact identity function ($h_{\text{out}} = h$). Gradients flow cleanly through arbitrarily deep networks at step 0 without exploding or vanishing.
- Q70. How does SDXL eliminate training crop artifacts via micro-conditioning?**
- Previous diffusion models discarded non-square images by center-cropping them to 512×512 , teaching the model that subjects are often cut in half at image borders. **SDXL Micro-Conditioning (Podell et al., 2023)**: During training, random crops are drawn from variable aspect-ratio images, and the exact pixel crop coordinates ($c_{\text{top}}, c_{\text{left}}$) along with original resolution ($h_{\text{orig}}, w_{\text{orig}}$) are embedded via Fourier features and injected into the UNet conditioning vector. At inference, setting ($c_{\text{top}} = 0, c_{\text{left}} = 0$) conditions the model to generate fully uncropped, centered compositions.
- LATENT DIFFUSION PITFALLS

 - ▶ **VAE Latent Scaling Factor**: Latent representations from the VAE encoder have non-unit variance ($\sigma \neq 1$). SD scales latents by constant factor 0.18215 (SDXL uses 0.13025) before adding noise: $z = 0.18215 \cdot \mathcal{E}(x)$. Forgetting this scaling factor leads to complete generation failure (snow noise).
 - ▶ **Latent VAE artifacts**: High spatial compression ($f = 8$) creates artifacts on fine typography and small faces. Face restorers (CodeFormer) or lower compression ($f = 4$) are required for micro-detail reconstruction.

19 Flow Matching, Rectified Flow & Flux.1

[Flow Matching · straight paths · velocity objective · Rectified Flow · Flux.1 12B · OT-CFM]



Straight Probability Paths. Diffusion trajectories curve stochastically through latent space, requiring small steps.

► **Constant Velocity Target:** $x_t = (1 - t)x_0 + tx_1 \implies v_t = \frac{dx_t}{dt} = x_1 - x_0$

► **Single-step Euler:** Straight paths make 1-to-4 step generation possible without distillation.

FLOW MATCHING VS DIFFUSION COMPARISON

Property	Standard Diffusion (DDPM/EDM)	Flow Matching / Rectified Flow
Trajectory	Curved probability paths	Straight linear trajectories ($x_t = (1 - t)x_0 + tx_1$)
Prediction Target	Noise ϵ_t or data x_0	Constant velocity vector field $v_t = x_1 - x_0$
Time Parameter	Discrete $t \in [1, 1000]$ or continuous σ	Normalized interval $t \in [0, 1]$
Sampling Step	Complex variance-adjusted ODE step	Simple linear update: $x_{t+\Delta t} = x_t + \Delta t \cdot v_\theta(x_t, t)$
Few-Step Quality	Blurry / degraded under 10 steps	Crisp, detailed generation in 4–8 steps

INTERVIEW Q&A | RECTIFIED FLOW MECHANICS & FLUX ARCHITECTURE

Q71. How does Rectified Flow straighten sampling paths, and what is Reflowing?

When training on random pairs of (x_0, x_1) (x_0 random noise, x_1 data image), trajectories can cross each other in state space. Where paths cross, the vector field must predict the average velocity of diverging paths, inducing local curvature. **Reflowing Procedure (2-Rectified Flow):**

1. Generate synthetic pairs $(x_0, \hat{x}_1)$ by simulating the trained 1-rectified flow ODE from $t = 0$ to $t = 1$.
2. Train a new flow matching model on these deterministic coupled pairs $(x_0, \hat{x}_1)$.

Because the pairs come from an ODE whose paths cannot cross (uniqueness of ODE solutions), the new coupling has less crossing and the learned paths are straighter; Liu et al. prove reflow never increases transport cost. In practice 2-rectified flow plus distillation gives good samples in 1 to 2 Euler steps.

Q72. What is Optimal Transport Flow Matching (OT-CFM)?

Tong et al. (2023) observe that standard flow matching pairs random Gaussian noise x_0 with random data x_1 independently, producing long, crossing trajectories. **Optimal Transport CFM:** Couples pairs (x_0, x_1) according to the optimal transport plan π minimizing dynamic kinetic energy:

$$\mathcal{W}_2^2(p_0, p_1) = \inf_{\pi} \int \|x_0 - x_1\|_2^2 d\pi(x_0, x_1)$$

Solve exact OT (Hungarian / EMD) within each mini-batch and repair noise with data by that plan. Paths cross less, training variance drops and fewer ODE steps are needed; the benefit shrinks in high dimensions where mini-batch OT is a rough approximation.

Q73. Walk through the 12B parameter architecture of Flux.1.

Flux.1 (Black Forest Labs) is an MM-DiT in the style of SD3, with no cross-attention:

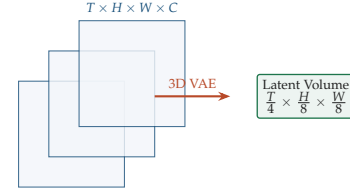
1. **Double Stream Blocks (19 blocks):** Image and text tokens keep *separate weights* (norms, QKV, MLP) but run *one joint attention* over the concatenated sequence, so each modality reads the other while keeping specialized parameters.
2. **Single Stream Blocks (38 blocks):** Concatenates text and visual tokens into a single unified residual stream, running bidirectional self-attention across all tokens simultaneously.
3. **Dual Text Conditioning:** T5-XXL token sequence (prompt structure, spelling) enters the text stream; the pooled CLIP ViT-L vector is added to the timestep embedding for adaLN modulation (global style).
4. **Rotary Positional Embeddings (2D RoPE):** Replaces fixed absolute coordinates, enabling training and inference across any aspect ratio and resolution up to 2.0 megapixels without tiling.

FLOW MATCHING GOTCHAS

- **Direction of t :** In diffusion literature, $t = 0$ is clean data and $t = T$ is pure noise. In Flow Matching literature, conventions are often **inverted**: $t = 0$ is pure Gaussian noise ($x_0 \sim \mathcal{N}(0, I)$) and $t = 1$ is target data (x_1). Check the boundary conditions of your solver ($0 \rightarrow 1$ vs $1 \rightarrow 0$).
- **CFG formulation in Flow Matching:** Naive CFG extrapolation on velocity fields can overshoot data manifolds. Flux uses guidance distillation or interval-constrained CFG to prevent color explosion.

20 Video Foundation Models: Spatio-Temporal DiT

[Factorized attention · 3D VAE · Space-Time DiT · Sora mechanics · Causal masking · VBench]



The Spatio-Temporal Explosion. Naive full 3D self-attention over video tensor $\mathbb{R}^{B \times C \times T \times H \times W}$ scales as $\mathcal{O}((T \cdot H \cdot W)^2)$, causing immediate out-of-memory crashes on GPU.

► **Factorized Attention:** Decouples spatial attention (within frame) from temporal attention (across time) $\rightarrow \mathcal{O}(T(HW)^2) + (HW)T^2$.

VIDEO REPRESENTATION PARADIGMS

Architecture	Mechanism	Trade-off
3D Conv (I3D)	Inflates 2D $K^2 \rightarrow 3D K^3$; repeats weights divided by K	Rigid temporal receptive field; high parameter overhead.
Factorized ViT (ViViT)	Alternates Spatial Self-Attn (HW tokens) and Temporal (T tokens)	Linear in T ; enables training on 32 – 64 frames on consumer clusters.
3D DiT (Sora)	Spatio-temporal 3D VAE (8×8 space, $4 \times$ time) + 3D DiT	Monotonic scaling; high fidelity; requires heavy compute clusters.

INTERVIEW Q&A | VIDEO DIFFUSION, 3D VAE & EVALUATION

Q74. How does a 3D Spatio-Temporal VAE compress video, and why is temporal downsampling tricky?

A 10-second 1080p video at 24 FPS contains 240 frames $\approx 1.5 \times 10^9$ raw values. A **3D Causal VAE** compresses data along all 3 dimensions:

- **Spatial:** 8×8 downsampling ($H/8, W/8$).
- **Temporal:** $4 \times$ downsampling ($T/4 \implies 60$ latent frames).

The Challenge: Pure 3D convolutions look into future frames, destroying causal streaming generation. Modern video VAEs enforce **causal temporal convolutions** (padding only on past frames), ensuring that encoding/decoding frame t never requires future frames $t + 1$.

Q75. How do modern video models (e.g. Sora, Movie Gen) achieve variable duration and aspect ratios?

Following the NaViT/Patch n' Pack paradigm:

1. **Spacetime Patches:** Video latents are partitioned into 3D spacetime cubes of size $p_t \times p_h \times p_w$ (e.g., $1 \times 2 \times 2$).
2. **Flattened Sequence:** Any video clip—regardless of resolution or duration—flattens into a 1D token sequence of length $N = \frac{T}{p_t} \cdot \frac{H}{p_h} \cdot \frac{W}{p_w}$.
3. **3D RoPE:** Position embeddings are decomposed into 3 rotary frequency components ($R_{\text{time}}, R_{\text{height}}, R_{\text{width}}$). Allows training on mixed datasets of vertical TikTok videos, widescreen 4K film clips, and images ($T = 1$) in the same training batches without cropping or padding.

Q76. Why is Fréchet Video Distance (FVD) flawed, and how does VBench fix it?

Flaws of FVD: FVD computes Fréchet distance on features from an I3D model pretrained on Kinetics-400. It is heavily biased toward low-resolution action recognition features, ignores temporal consistency (adjacent flickering frames can still produce identical average feature distributions), and penalizes stylistic artistic videos. **VBench (Huang et al., 2024):** Comprehensive evaluation decomposed into 16 distinct sub-benchmarks:

- **Temporal Quality:** Temporal flickering, motion smoothness, dy-

namic degree, subject consistency, background consistency.

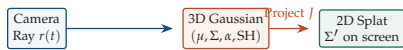
- **Video-Text Alignment:** Action accuracy, scene transitions, multiple objects, spatial relationships. Evaluates using fine-tuned specialized vision models and tracking algorithms, providing granular diagnosis of video failure modes.

VIDEO GENERATION TRAPS

- **Flicker / Temporal Incoherence:** Training video diffusion without causal frame attention or 3D temporal discriminator losses causes adjacent frames to sample diverging textures ("boiling" noise artifacts).
- **Autoregressive drift:** Extending video length by feeding the last generated frame as prompt for the next chunk suffers from error compounding; latents blur or explode after 10 – 15 seconds without noise re-injection.

21 3D Vision: NeRF to 3D Gaussian Splatting

[NeRF · Volume rendering · Instant-NGP · 3DGS covariance · Tile rasterization · SH radiance]



Implicit vs Explicit 3D. NeRF represents scenes implicitly via continuous MLPs. 3D Gaussian Splatting (3DGS) represents scenes explicitly via millions of anisotropic 3D Gaussians.

- **3DGS Real-Time Rendering:** Replaces expensive volumetric MLP ray-marching with GPU tile-based rasterization at > 100 FPS.

NERF VS INSTANT-NGP VS 3DGS

	NeRF (2020)	Instant-NGP (2022)	3DGS (2023)
Scene	8-layer MLP, 256 wide	Hash grids + tiny MLP	Millions of explicit Gaussians
Render	Ray march, ~192 MLP calls/ray	Ray march, occupancy-grid skipping	Sort + rasterize splats
Train	1–2 days	Seconds to minutes	~30 min
Speed	Seconds per frame	~10–60 FPS	100+ FPS at 1080p
Memory	~5 MB	~50 MB	200 MB–1 GB
Weak spot	Speed	Floater, aliasing	Memory, needles, needs SfM init

INTERVIEW Q&A | 3D REPRESENTATION, NERF & GAUSSIAN RASTERIZATION

Q77. Walk through the Volume Rendering equation in NeRF and its discrete quadrature.

NeRF casts a ray $r(t) = \mathbf{o} + t\mathbf{d}$ from camera origin $\mathbf{o}$ along direction $\mathbf{d}$. The continuous color integral between near and far bounds $[t_n, t_f]$ is:

$$C(r) = \int_{t_n}^{t_f} T(t) \cdot \sigma(r(t)) \cdot c(r(t), \mathbf{d}) dt$$

$$\text{where } T(t) = \exp\left(-\int_{t_n}^t \sigma(r(s)) ds\right)$$

$T(t)$ is **transmittance** (probability the ray traverses from t_n to t without hitting particles). **Discrete Numerical Quadrature:** Sample N points along the ray:

$$C(r) \approx \sum_{i=1}^N T_i \cdot \alpha_i \cdot c_i \quad \text{with } \alpha_i = 1 - \exp(-\sigma_i \delta_i), \quad T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

Evaluating this on a 1080p image requires $\approx 200\text{M}$ MLP forward passes, bounding vanilla NeRF to several seconds per frame.

Q78. How does Instant-NGP accelerate NeRF training from days to seconds?

Müller et al. identify that large MLPs are slow to evaluate and suffer from spectral bias on high spatial frequencies. **Instant-NGP (Multiresolution Hash Encoding):**

1. Discretizes 3D space into $L = 16$ hierarchical grid levels with geometric progression of resolutions N_l .
2. Maps voxel corners to a fixed-size hash table of size $T = 2^{19}$ using spatial hashing: $h(\mathbf{x}) = \left(\bigoplus_{i=1}^3 x_i \pi_i\right) \pmod T$.
3. Interpolates feature vectors ($d = 2$ per level) trilinearly within the voxel.

4. Feeds concatenated features ($16 \times 2 = 32$ -dim) into a tiny 2-layer MLP (64 neurons) implemented with fully fused CUDA kernels. Trains high-quality radiance fields in **5 seconds**.

Q79. Why does NeRF need positional encoding, and what is the coarse-to-fine sampler?

MLPs have **spectral bias**: fed raw (x, y, z) they learn smooth, blurry functions. NeRF maps each coordinate through

$$\gamma(p) = (\sin(2^0 \pi p), \cos(2^0 \pi p), \dots, \sin(2^{L-1} \pi p), \cos(2^{L-1} \pi p))$$

with $L = 10$ for position (60-d) and $L = 4$ for view direction (24-d), letting the MLP fit high frequencies (the same idea as Fourier features). **Hierarchical sampling:** a coarse network evaluates 64 stratified samples per ray; its weights $w_i = T_i \alpha_i$ form a piecewise-constant PDF; 128 more samples are drawn by inverse-CDF sampling near surfaces and the fine network evaluates all 192. Density depends only on position; color also on view direction, which models specular effects.

3D VISION PITFALLS

- **Needle and Floater Artifacts in 3DGS:** Excessively large Gaussians in under-constrained background regions create needle-like spikes. Periodic **opacity reset** (forcing all $\alpha \rightarrow 0$ every 3000 steps) prunes spurious floaters.
- **Camera coordinate conventions:** OpenCV uses (X right, Y down, Z forward); OpenGL/Blender uses (X right, Y up, Z backward). Mixing coordinate conventions flips camera rays and produces completely black renders.

22 3D Gaussian Splatting: Math, Rasterizer & SH

[3D Covariance Σ · EWA Jacobian projection · Spherical Harmonics · 64-bit Radix Sort · Adaptive density control]

PARAMETERS PER 3D GAUSSIAN PRIMITIVE

Attribute	Shape	Type	Mathematical Representation / Activation
Position (μ)	3 floats	$\mathbb{R}^3$	World coordinates (x, y, z)
Scale (s)	3 floats	$\mathbb{R}^3$	Semi-axis lengths via exponential $\exp(s) > 0$
Rotation (q)	4 floats	$\mathbb{H}$	Unit quaternion $(w, x, y, z) / \ q\ $
Opacity (α)	1 float	$[0, 1]$	Blending density via sigmoid $\sigma(\alpha)$
Color / SH	48 floats	$\mathbb{R}^{3 \times 16}$	Spherical Harmonics degree 3 (16×3 RGB)

Total Memory Footprint:

59 floats $\times$ 4 bytes $\approx$

236 bytes per Gaussian. At

3M Gaussians $\Rightarrow \sim 708$ MB.

INTERVIEW Q&A | 3D COVARIANCE PARAMETERIZATION & EWA PROJECTION

Q80. Derive why 3DGS parameterizes covariance as $\Sigma = RSS^T R^T$ and how it projects into 2D screen space.

A valid covariance matrix must be **symmetric positive semi-definite** ($\Sigma \succeq 0$). Directly optimizing $\Sigma \in \mathbb{R}^{3 \times 3}$ violates positive definiteness during gradient descent, causing imaginary eigenvalues and catastrophic collapse.

Decomposition into Scale and Rotation:

- $\mathbf{S} = \text{diag}(\exp(s_x), \exp(s_y), \exp(s_z))$ guarantees non-zero real scaling.
- $\mathbf{R} \in \text{SO}(3)$ is computed from normalized unit quaternion $\mathbf{q} = (w, x, y, z) / \sqrt{w^2 + x^2 + y^2 + z^2}$.
- Thus $\Sigma = \mathbf{R} \mathbf{S} \mathbf{S}^T \mathbf{R}^T$ is mathematically guaranteed to be symmetric positive semi-definite.

2D EWA Projection (Zwicker et al.): Given camera viewing transformation $\mathbf{W}$ (world-to-camera matrix) and projective transformation, the 2D projected covariance $\Sigma' \in \mathbb{R}^{2 \times 2}$ is given by:

$$\Sigma' = \mathbf{J} \mathbf{W} \Sigma \mathbf{W}^T \mathbf{J}^T + 0.3 \cdot \mathbf{I}_{2 \times 2}$$

where $\mathbf{J}$ is the Jacobian of the affine approximation of projective transformation evaluated at camera coordinates (t_x, t_y, t_z) :

$$\mathbf{J} = \begin{bmatrix} \frac{f_x}{t_z} & 0 & -\frac{f_x t_x}{t_z^2} \\ 0 & \frac{f_y}{t_z} & -\frac{f_y t_y}{t_z^2} \end{bmatrix}$$

The term $0.3 \cdot I_{2 \times 2}$ acts as a low-pass Gaussian blur filter, preventing aliasing artifacts when a splat projects to an area smaller than a single pixel.

Q81. Explain how Spherical Harmonics model view-dependent specular reflections in 3DGS.

Lambertian surfaces reflect light uniformly in all directions; non-Lambertian surfaces (metals, mirrors, glass) reflect different colors depending on viewing direction $\mathbf{d} = \frac{\mathbf{x}-\mathbf{c}}{\|\mathbf{x}-\mathbf{c}\|}$.

Spherical Harmonics (SH) Expansion: Color $c(\mathbf{d}) \in \mathbb{R}^3$ is modeled as a linear combination of orthonormal basis functions $Y_l^m(\mathbf{d})$ on the unit sphere:

$$c(\mathbf{d}) = \sum_{l=0}^{l_{\max}} \sum_{m=-l}^l k_l^m Y_l^m(\mathbf{d})$$

- **Degree 0** ($l = 0, 1$ basis): $Y_0^0 = \frac{1}{2} \sqrt{\frac{1}{\pi}}$. Represents constant ambient/diffuse color (1 coeff $\times 3 = 3$ floats).
- **Degree 1** ($l = 1, 3$ bases): Directional gradient (4 total coeffs $\times 3 = 12$ floats).
- **Degree 2** ($l = 2, 5$ bases): Quadric reflection lobes (9 total coeffs $\times 3 = 27$ floats).
- **Degree 3** ($l = 3, 7$ bases): Sharp specular highlights (16 total coeffs $\times 3 = 48$ floats).

INTERVIEW Q&A | TILE-BASED TILE RASTERIZER & ADAPTIVE DENSIFICATION

Q82. Step through the GPU tile-based rasterization pipeline that enables 100+ FPS real-time rendering.

1. **Screen Tiling:** Divides display resolution into non-overlapping 16×16 pixel tiles.
2. **Culling & Bounding:** Evaluates 99% confidence ellipse (3σ) for each 2D projected Gaussian $\blacksquare$; discards splats outside viewing frustum.
3. **Tile Assignment & 64-bit Key:** For each tile a Gaussian overlaps, generates a duplicate 64-bit integer key:

$$\text{Key} = [\text{Tile ID (32-bit)} \mid \text{Viewspace Depth } z \text{ (32-bit)}]$$

4. **GPU Radix Sort:** A single parallel GPU Radix Sort orders all key-value pairs by Tile ID, and within each tile, strictly front-to-back by depth z .
5. **Tile Rasterization Warp:** One thread block processes one tile; threads load sorted Gaussians into shared memory and blend colors front-to-back:

$$C_{\text{pixel}} = \sum_{i=1}^N c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j), \quad \alpha_i = \sigma_i \exp\left(-\frac{1}{2} \Delta^T \blacksquare'^{-1} \Delta\right)$$

6. **Early Ray Termination:** Accumulates transmittance $T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$; once $T_i < 0.0001$, rendering terminates immediately for that pixel.

Q83. How does Adaptive Density Control decide whether to Clone or Split a 3D Gaussian?

During optimization, 3DGS tracks the average view-space position gradient magnitude $\nabla_p = \|\frac{\partial \mathcal{L}}{\partial \mu_{2D}}\|_2$. If $\nabla_p > \tau_{\text{pos}} = 0.0002$, the region is improperly reconstructed:

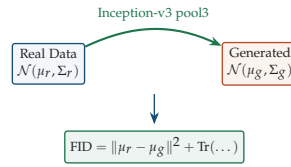
- **If Scale is Small** ($\max(s) < \tau_{\text{scale}}$): Represents an **under-reconstructed** region (e.g. missing thin bicycle spokes or leaf geometry). The Gaussian is **cloned** (duplicated) and shifted along the gradient direction.
- **If Scale is Large** ($\max(s) > \tau_{\text{scale}}$): Represents an **over-reconstructed** region (a huge Gaussian trying to cover too much space). The Gaussian is **split** into two smaller Gaussians, each scaled down by factor $1.6 \times$.
- **Opacity Reset & Pruning:** Every 3000 iterations, opacities are reset to near-zero ($\alpha \leftarrow \min(\alpha, 0.01)$); Gaussians unable to rebuild opacity ($\alpha < 0.005$) are permanently pruned.

3DGS OPTIMIZATION PITFALLS

- **Needle Floater Artifacts:** Large background Gaussians very close to the camera form huge translucent "needle" streaks when rotated. Solution: Clamp minimum depth z_{near} and penalize maximum spatial scale $\|s\|_\infty$ in the loss.
- **Memory Thashing during Densification:** Without periodic opacity pruning ($\alpha < 0.005$), Gaussian count explodes from 1M $\rightarrow$ 12M, exhausting GPU VRAM during training.

23 Vision Benchmarks, Metrics & Evaluation

[ImageNet-C · COCO mAP · Panoptic PQ · FID 50k · PSNR / SSIM / LPIPS]



The Evaluation Landscape. Visual evaluation spans categorical recognition, dense pixel geometry, perceptual fidelity, and multimodal reasoning.

- **COCO mAP:** Evaluated across 10 IoU thresholds from 0.50 to 0.95 with 0.05 step size.
- **FID:** Measures 2-Wasserstein distance between multivariate Gaussian features in Inception space.

KEY VISION BENCHMARKS & THRESHOLDS

Domain	Benchmark	Standard Metric
Classification	ImageNet-1K, ImageNet-A / R/C	Top-1 Acc (%) / Mean Corruption Error (mCE)
Detection	MS COCO (80 classes)	mAP _{50:95} , AP _S , AP _M , AP _L
Segmentation	ADE20K (semantic), Cityscapes	Mean IoU (mIoU), Panoptic Quality (PQ = SQ $\times$ RQ)
VLM Reasoning	MMBench, MathVista, DocVQA	Overall Accuracy, OCR-F1 score
Hallucination	POPE (Random, Popular, Adversarial)	Binary F1-score & Yes-ratio
Generative	ImageNet 512 ² , COCO 30k	FID (< 3.0 frontier), CLIP Score (> 30.0)
3D Quality	NeRF Synthetic, Tanks & Temples	PSNR (> 32 dB), SSIM (> 0.95), LPIPS (< 0.05)

INTERVIEW Q&A | METRIC FORMULATIONS & EVALUATOR PITFALLS

Q84. State the Fréchet Inception Distance (FID) formula and explain its vulnerabilities.

Given feature representations from the 2048-dimensional pool3 layer of Inception-v3:

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \text{Tr}\left(\blacksquare_r + \blacksquare_g - 2(\blacksquare_r \blacksquare_g)^{1/2}\right)$$

This is the 2-Wasserstein distance between two continuous Gaussians $\mathcal{N}(\mu_r, \Sigma_r)$ and $\mathcal{N}(\mu_g, \Sigma_g)$. **Vulnerabilities:**

1. **Sample Size Sensitivity:** FID is biased upward with small sample counts; comparing FID requires evaluating on exactly 50,000 generated samples ($N = 50k$).
2. **Inception Bias:** Inception-v3 was trained on ImageNet and is blind to human anatomy, facial distortion, text spelling, and high-frequency noise.
3. **JPEG and Resizing artifacts:** Compressing images or using bilinear rather than bicubic resizing alters FID by up to ± 5 points without perceptible image differences.

Q85. PSNR vs SSIM vs LPIPS: what does each measure, and when does each lie?

PSNR = $10 \log_{10} \frac{\text{MAX}^2}{\text{MSE}}$ dB. Pixel-wise; a 1-pixel shift or slight blur can cost several dB while looking identical, and blurry averages score well. $+1 \text{ dB} \approx 21\%$ lower MSE.

SSIM compares local means, variances and covariance (luminance, contrast, structure) in 11×11 Gaussian windows:

$$\text{SSIM} = \frac{(2\mu_x \mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

Better on structure, still insensitive to texture realism.

LPIPS: distance between unit-normalized deep features (AlexNet/VGG) with learned per-channel weights, trained on human 2AFC judgements. Tracks perceived similarity best; lower is better.

Rule: report all three for reconstruction (NeRF/3DGS, super-resolution, VAE); a GAN-sharpened output can lose PSNR but win LPIPS. For generation without a reference image, use FID/CLIPScore instead.

Q86. How is Panoptic Quality (PQ) decomposed into SQ and RQ?

Kirillov et al. decompose Panoptic Quality as $\text{PQ} = \text{SQ} \times \text{RQ}$:

$$\text{SQ} = \frac{\sum_{(p,g) \in TP} \text{IoU}(p,g)}{|TP|}, \quad \text{RQ} = \frac{|TP|}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}$$

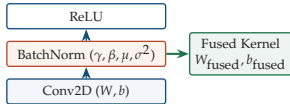
- **Segmentation Quality (SQ):** Measures average boundary overlap purely among true-positive matched segments (IoU > 0.5).
- **Recognition Quality (RQ):** Standard F1-score measuring detection/-classification accuracy of segments regardless of pixel tightness. Decouples mask boundary quality from false-positive detection errors.

BENCHMARKING TRAPS

- **ImageNet-C Corruption Leakage:** Using Gaussian blur or noise during training data augmentation contaminates ImageNet-C evaluation, invalidating reported robustness metrics.
- **CLIPScore Gamification:** Generating images with giant repeated text labels exploits CLIP's text-matching bias, yielding high CLIP-Score despite awful visual image quality.

24 Production Vision, Optimization & Edge Deployment

[Preprocessing · Conv-BN fusion · TensorRT · INT8 PTQ · Edge latency budgets · ByteTrack]



The Zero-Cost Fusion. BatchNorm parameters are linear and static at test time.

- **Folding into Weights:** $W_{\text{fused}} = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} W$, $b_{\text{fused}} = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} (b - \mu) + \beta$
- Eliminates 2× memory roundtrips to GPU DRAM per convolution block.

EDGE INFERENCE LATENCY BUDGETS (TARGET: REAL-TIME)

Application	Budget	Recommended Model Architecture
Robotics	/ 16.6 ms	YOLOv8-Nano / MobileNetV4 / TensorRT INT8
Drones		
Auto Driving	33.3 ms	RegNet / ConvNeXt-T / BEVDet on NVIDIA Orin
AR / VR	11.1 ms	3D Gaussian Splatting with Metal/Vulkan
Mobile Phone	33.3 ms	Apple CoreML Neural Engine / INT8 VLM

INTERVIEW Q&A | PRODUCTION PIPELINE DYNAMICS & OPTIMIZATIONS

Q87. Derive the exact mathematical equations for fusing Conv2D and BatchNorm.

A Conv2D layer computes $x_{\text{conv}} = Wx + b$. BatchNorm normalizes and scales: $y = \gamma \frac{x_{\text{conv}} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$. Substituting x_{conv} directly into the BatchNorm formula:

$$y = \gamma \frac{(Wx + b) - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

$$= \left(\frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} W \right) x + \left(\frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} (b - \mu) + \beta \right)$$

Define fused weights and bias:

$$W_{\text{fused}} = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} W$$

$$b_{\text{fused}} = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} (b - \mu) + \beta$$

W_{fused} is a standard $C_{\text{out}} \times C_{\text{in}} \times K \times K$ tensor and b_{fused} is a C_{out} vector. The BatchNorm operation and its memory allocation are completely eliminated at test time.

Q88. Why can BGR vs RGB channel order silently cost tens of accuracy points, and how is it audited?

OpenCV reads image buffers from disk or webcam in **BGR** format by default. PyTorch and Torchvision backbones (ResNet, ViT, CLIP) are trained exclusively on **RGB** arrays. Feeding BGR images into an RGB-trained model swaps the red and blue channels: skin tones become bright cyan/blue; blue skies become reddish-orange.

Auditing Rule: Always assert channel ordering at pipeline entry: `img = cv2.cvtColor(bgr_frame, cv2.COLOR_BGR2RGB)`. Add a static unit test asserting that average red channel intensity of an Apple image exceeds blue intensity.

Q89. Walk through INT8 Post-Training Quantization (PTQ) with KL Calibration.

FP32 weights and activations are mapped to 8-bit integers $[-128, 127]$ via scaling factor S : $x_{\text{int8}} = \text{clip}(\text{round}(x/S), -128, 127)$. For activations with long-tail outlier distributions, clipping at the maximum absolute value wastes precision on rare extremes.

NVIDIA TensorRT KL Calibration:

1. Collect activation histograms over a calibration dataset (~ 500 representative images).
2. For candidate clipping thresholds T : compute quantized and reconstructed distribution P and Q .
3. Choose threshold T^* minimizing Kullback-Leibler divergence:

$$T^* = \arg \min_T D_{\text{KL}}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Retains 99.5% of FP32 accuracy while cutting memory bandwidth by 4× and doubling compute throughput on Tensor Cores.

Q90. How does ByteTrack track multi-object targets across occlusions in real time?

Traditional tracking-by-detection algorithms (SORT, DeepSORT) discard low-confidence detections ($s < 0.6$), losing targets during occlusions. **ByteTrack (Zhang et al., 2022):**

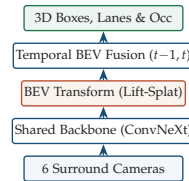
1. First Association: Matches high-score detections ($s \geq 0.6$) with existing tracklets using Kalman filter motion predictions and bounding box IoU.
2. Second Association: Takes the unmatched tracklets and matches them with **low-score detections** ($0.1 \leq s < 0.6$). Low-confidence detections typically represent partially occluded or blurred objects. Re-associating them preserves target identity across obstacles at > 60 FPS with zero neural network re-evaluation.

DEPLOYMENT TRAPS

- **Letterboxing vs Stretching:** Resizing non-square camera feeds (1920×1080) to 640×640 via simple stretching warps aspect ratios, corrupting bounding box regression and object detection confidence. Letterbox with symmetric grey borders (114, 114, 114).
- **Synchronous video loops:** Calling `cap.read()` synchronously inside the inference loop blocks GPU execution waiting for camera USB/MIPI driver I/O. Use a double-buffered background thread for frame acquisition.

25 Vision System Design: Autonomous Driving to Visual RAG

[System design framework · BEV driving perception · Visual Document RAG · Sub-second GenAI API · ColPali]



- The 6-Step System Design Recipe.**
1. Clarify SLOs (latency, FPS, compute tier, accuracy).
 2. Data engine (collection, active learning, auto-labeling).
 3. Architecture (backbone, 2D-to-3D projection, heads).
 4. Training setup (loss balance, distributed recipe).
 5. Serving (compilation, INT8 PTQ, pipelined batching).
 6. Telemetry & OOD drift detection.

FRONTIER SYSTEM ARCHITECTURES

System	Core Bottleneck	Winning Architectural Decision
Autonomous Driving	Multi-camera occlusion; 3D spatial alignment	Bird's-Eye-View (BEVFormer): 2D feature maps lifted to unified 3D/BEV coordinate grid via deformable cross-attention; fused temporally across past frames.
Visual RAG	Doc Dense OCR charts, table layouts, tiny fonts	ColPali / Dynamic VLM: Replaces lossy text OCR pipelines with late-interaction patch embeddings; computes token-to-patch max-sim directly over document images.
Sub-Second Diffusion	Iterative multi-step sampling latency	Step Distillation (ADD / LCM) + TensorRT: Distills 50-step diffusion to 4-step rectified flow; FP8 engine executes in < 400 ms on single L40S.

INTERVIEW Q&A | ARCHITECTURE DEEP-DIVES

Q91. Design an end-to-end multi-camera 3D perception system for an autonomous vehicle.

Requirements: 6 surround cameras (1920×1080), 30 FPS, latency budget ≤ 33 ms on an automotive SoC (e.g., NVIDIA DRIVE Orin, 275 TOPS). Output: 3D bounding boxes $(x, y, z, w, l, h, \theta)$, velocity vectors (v_x, v_y) , lane topologies. **Pipeline Architecture:**

- Image Backbone:** ConvNeXt-Tiny with shared weights across all 6 views at half resolution (960×540), outputting multi-scale feature maps.
- View Transformation:** BEVFormer maintains a persistent 2D BEV grid ($200 \times 200 \times 1\text{m}$). Uses spatial cross-attention: 3D grid points project to 2D image coordinates via camera extrinsics $[R|t]$ and intrinsics K ; deformable attention queries sample image features.
- Temporal Recurrence:** Warps past BEV features ($t-1$) into current frame coordinate system using ego-motion odometry; concatenates along channels.
- Task Heads:** Anchor-free query detectors (DETR-style 3D boxes) and semantic occupancy grid heads.
- Inference Engine:** Exported to TensorRT with FP16/INT8 mixed precision, operating in 28 ms.

Q92. Design a visual document understanding system (Visual RAG) for unstructured invoices and financial charts.

Failure of Traditional OCR: Standard Tesseract/Texttract extracts raw text strings, destroying table column alignments, multi-column reading orders, and visual diagrams. **Modern Vision-Centric Solution:**

- Page Indexing:** Render PDF pages directly as 150 DPI images.
- Retrieval Engine (ColPali):** Feed page images into PaliGemma-3B (SigLIP encoder) and keep every output patch embedding projected to 128-d ($32 \times 32 = 1024$ patches/page). Store in multi-vector index. User text queries compute Late Interaction:

$$S(q, d) = \sum_{i=1}^{|q|} \max_{j=1}^{|d|} \left(\mathbf{q}_i^T \mathbf{d}_j \right)$$

On the ViDoRe benchmark ColPali reaches ~ 81 nDCG@5 versus ~ 65 – 67 for OCR/captioning + text-retriever pipelines, at the cost of ~ 1030 vectors per page.

- Reader / Reasoner:** Selected top-3 document page images are fed into a high-resolution VLM (Qwen2-VL) with native dynamic resolution, generating structured JSON with source bounding box coordinates.

Q93. How does Adversarial Diffusion Distillation (ADD) compress 50-step diffusion into 1 to 4 steps?

Sauer et al. combine score distillation with an adversarial objective to train a student network G_θ :

$$\mathcal{L}_{\text{ADD}} = \mathcal{L}_{\text{adv}}(G_\theta, D_\phi) + \lambda_{\text{score}} \mathcal{L}_{\text{score}}(G_\theta, \text{Teacher})$$

- **Adversarial Loss:** Discriminator D_ϕ forces single-step generated images to lie strictly on the manifold of natural images, avoiding blurriness.
- **Score Distillation Loss:** Uses a frozen pretrained SDXL model as teacher; computes gradients in diffusion latent space to align outputs with the text prompt. Enables real-time text-to-image synthesis at 4 steps (< 400 ms per megapixel frame).

SYSTEM DESIGN PITFALLS

- **Ignoring sensor synchronization:** Driving systems where front and side cameras are not hardware genlocked (frame timestamp delta > 5 ms) produce double-vision artifacts and phantom obstacles in BEV space.
- **Cold start VLM latency:** High-resolution VLM prefill takes up to 500 ms for large document images. Mitigate with visual token caching (caching vision encoder outputs across repeated queries on the same document).

26 Implement from Scratch: Vision Code You Write Blind

[Conv2D im2col · ResNet Bottleneck · PatchEmbed · 2D RoPE · CLIP InfoNCE · IoU & NMS · UNet Block · Flow Matching]

Minimal PyTorch snippets (assume `import torch, torch.nn as nn, torch.nn.functional as F`). Shapes in comments. Interviewers look for: dimensions, channel positions, broadcasting, and numerical stability.

1. Conv2D via Unfold (im2col formulation)

```
def conv2d_im2col(x, weight, b=None, stride=1, padding=0):
    # x: (B, Cin, H, W) ; weight: (Cout, Cin, K, K)
    B, Cin, H, W = x.shape; Cout, _, K, _ = weight.shape
    H_out = (H - K + 2 * padding) // stride + 1
    W_out = (W - K + 2 * padding) // stride + 1
    cols = F.unfold(x, K, padding=padding, stride=stride) # (B, Cin*K*K, L)
    out = weight.view(Cout, -1) @ cols # (B, Cout, L)

    out = out.view(B, Cout, H_out, W_out)
    return out if b is None else out + b.view(1, Cout, 1, 1)
```

2. ResNet Bottleneck Block with Shortcut

```
class Bottleneck(nn.Module):
    def __init__(self, in_c, mid_c, out_c, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_c, mid_c, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(mid_c)
        self.conv2 = nn.Conv2d(mid_c, mid_c, 3, stride=stride, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(mid_c)
        self.conv3 = nn.Conv2d(mid_c, out_c, 1, bias=False)
        self.bn3 = nn.BatchNorm2d(out_c)
        self.shortcut = nn.Sequential(
            nn.Conv2d(in_c, out_c, 1, stride=stride, bias=False),
            nn.BatchNorm2d(out_c)
        ) if (stride != 1 or in_c != out_c) else nn.Identity()

    def forward(self, x):
        res = self.shortcut(x)
        out = F.relu(self.bn1(self.conv1(x)))
        out = F.relu(self.bn2(self.conv2(out)))
        out = self.bn3(self.conv3(out))
        return F.relu(out + res)
```

3. ViT Patch Partition & Linear Projection

```
class PatchEmbed(nn.Module):
    def __init__(self, img_size=224, patch_size=16, in_c=3, embed_dim=768):
        super().__init__()
        self.n_patches = (img_size // patch_size) ** 2
        self.proj = nn.Conv2d(in_c, embed_dim, kernel_size=patch_size, stride=patch_size)

    def forward(self, x):
        # x: (B, 3, H, W) -> (B, embed_dim, H/P, W/P) -> (B, N, embed_dim)
        return self.proj(x).flatten(2).transpose(1, 2)
```

4. 2D Rotary Position Embeddings (2D RoPE)

```
def apply_2d_rope(x, pos_h, pos_w, dim=64, base=10000.0):
    # x: (B, N, H_heads, D) ; pos_h: (N,) ; pos_w: (N,)
    half_d = dim // 2
    inv_freq = base ** (-torch.arange(0, half_d, 2).float() / half_d).to(x.device)
    sin_h = (pos_h[:, None] * inv_freq[None]).sin() # (N, dim/4)
    cos_h = (pos_h[:, None] * inv_freq[None]).cos()
    sin_w = (pos_w[:, None] * inv_freq[None]).sin()
    cos_w = (pos_w[:, None] * inv_freq[None]).cos()
    sin_2d = torch.cat([sin_h, sin_w], dim=-1) # (N, 1, dim/2)
    cos_2d = torch.cat([cos_h, cos_w], dim=-1) # (N, 1, dim/2)
    x1, x2 = x[:, :, :dim:2], x[:, :, dim:2]
    rot = torch.stack([x1*cos_2d - x2*sin_2d, x1*sin_2d + x2*cos_2d], dim=-1).flatten(-2)
    return torch.cat([rot, x[:, :, dim:]], dim=-1)
```

5. Symmetric InfoNCE Loss (CLIP Dual-Encoder)

```
def clip_infonce_loss(image_emb, text_emb, logit_scale):
    # image_emb: (B, D) ; text_emb: (B, D)
    I = F.normalize(image_emb, p=2, dim=-1)
    T = F.normalize(text_emb, p=2, dim=-1)
    sim = (I @ T.T) * logit_scale.exp().clamp(max=100.0) # (B, B)
    labels = torch.arange(len(I), device=I.device)
    loss_i2t = F.cross_entropy(sim, labels)
    loss_t2i = F.cross_entropy(sim.T, labels)
    return 0.5 * (loss_i2t + loss_t2i)
```

6. Vectorized Bounding Box IoU & Greedy NMS

```
def box_iou(b1, b2): # b1: (N, 4), b2: (M, 4) in (x1, y1, x2, y2)
    t1 = torch.max(b1[:, None, :2], b2[None, :, :2])
    br = torch.min(b1[:, None, 2:], b2[None, :, 2:])
    inter = (br - t1).clamp(min=0).prod(dim=-1)
    area1 = (b1[:, 2] - b1[:, 0]) * (b1[:, 3] - b1[:, 1])
    area2 = (b2[:, 2] - b2[:, 0]) * (b2[:, 3] - b2[:, 1])
    return inter / (area1[:, None] + area2[None, :] - inter + 1e-7)
```

```
def greedy_nms(bboxes, scores, iou_thresh=0.5):
    order = scores.argsort(descending=True)
    keep = []
    while len(order) > 0:
        i = order[0].item()
        keep.append(i)
        if len(order) == 1: break
        ious = box_iou(bboxes[i:i+1], bboxes[order[1:]] [0])
        order = order[1:] [ious <= iou_thresh]
    return torch.tensor(keep, dtype=torch.long)
```

7. UNet Double-Convolution Block with Skip Connection

```
class UNetBlock(nn.Module):
    def __init__(self, in_c, out_c):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_c, out_c, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_c), nn.ReLU(inplace=True),
            nn.Conv2d(out_c, out_c, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_c), nn.ReLU(inplace=True)
        )
    def forward(self, x, skip=None):
        if skip is not None:
            diff_h = skip.size(2) - x.size(2)
            diff_w = skip.size(3) - x.size(3)
            x = F.pad(x, [diff_w//2, diff_w - diff_w//2, diff_h//2,
                diff_h - diff_h//2])
            x = torch.cat([skip, x], dim=1)
        return self.conv(x)
```

8. Rectified Flow Matching Loss & Euler Step

```
def flow_matching_loss(model, x_1): # x_1: target data (B, C, H, W)
    x_0 = torch.randn_like(x_1) # x_0: source noise
    t = torch.rand(len(x_1), 1, 1, 1, device=x_1.device)
    x_t = (1.0 - t) * x_0 + t * x_1 # straight interpolation path
    target_v = x_1 - x_0 # constant velocity target
    pred_v = model(x_t, t.squeeze())
    return F.mse_loss(pred_v, target_v)

@torch.no_grad()
def sample_flow_euler(model, shape, n_steps=10, device='cuda'):
    x = torch.randn(shape, device=device) # start at noise (t=0)
    dt = 1.0 / n_steps
    for step in range(n_steps):
        t = torch.full((shape[0],), step * dt, device=device)
        v = model(x, t)
        x = x + dt * v # linear Euler update
    return x # final data at t=1
```

27 From-Scratch Vision Training Loops

[SimCLR loop · DDPM training step · DDIM sampler · AnyRes slicing]

Minimal PyTorch training and sampling loops. Focus on loss formulation, data shapes, and clean tensor operations.

1. SimCLR Contrastive Self-Supervised Step

```
import torch, torch.nn as nn, torch.nn.functional as F

def simclr_train_step(model, x1, x2, optimizer, tau=0.1):
    # x1, x2: Two augmented views of shape (B, 3, H, W)
    optimizer.zero_grad(set_to_none=True)
    B = x1.size(0)
    z1 = F.normalize(model(x1), dim=-1) # (B, D)
    z2 = F.normalize(model(x2), dim=-1) # (B, D)
    z = torch.cat([z1, z2], dim=0) # (2B, D)

    # Cosine similarity matrix scaled by temperature tau
    sim = torch.mm(z, z.t()) / tau # (2B, 2B)
    mask = torch.eye(2 * B, dtype=torch.bool, device=z.device)
    sim.masked_fill_(mask, -1e9) # Mask diagonal self-similarity

    # Positive targets: view 1 matches view 2
    targets = torch.cat([torch.arange(B, 2*B), torch.arange(0, B)]).to(z.device)
    loss = F.cross_entropy(sim, targets)
    loss.backward(); optimizer.step()
    return loss.item()
```

2. DDPM Noise-Prediction Training Step

```
def ddpm_train_step(net, x0, alpha_bar, optimizer):
    # x0: clean image (B, C, H, W); alpha_bar: cumulative schedule (T,)
    B = x0.size(0)
    optimizer.zero_grad(set_to_none=True)
    t = torch.randint(0, len(alpha_bar), (B,), device=x0.device)
    eps = torch.randn_like(x0)

    a_bar_t = alpha_bar[t].view(B, 1, 1, 1)
    xt = torch.sqrt(a_bar_t) * x0 + torch.sqrt(1.0 - a_bar_t) * eps

    eps_pred = net(xt, t)
    loss = F.mse_loss(eps_pred, eps)
    loss.backward(); optimizer.step()
    return loss.item()
```

3. Deterministic DDIM Sampler ($\eta = 0$)

```
@torch.no_grad()
def ddim_sample(net, shape, alpha_bar, n_steps=50, device='cuda'):
    # alpha_bar: (T,) cumulative product; net predicts eps
    T = len(alpha_bar)
    ts = torch.linspace(T - 1, 0, n_steps, device=device).long()
    x = torch.randn(shape, device=device) # x_T
    for i, t in enumerate(ts):
        a_t = alpha_bar[t]
        a_prev = alpha_bar[ts[i + 1]] if i + 1 < n_steps else torch.
            tensor(1.0, device=device)
        eps = net(x, t.expand(shape[0]))
        x0 = (x - (1 - a_t).sqrt() * eps) / a_t.sqrt() # predicted
            clean image
        x = a_prev.sqrt() * x0 + (1 - a_prev).sqrt() * eps # jump to
            t_prev
    return x
```

4. VLM AnyRes Dynamic Grid Partitioning

```
def anyres_slice(img, grid_pinpoints=[(1,1), (1,2), (2,1), (2,2)],
    patch_size=336):
    W, H = img.size
    best_grid = (1, 1); min_dist = float("inf")
    for (gw, gh) in grid_pinpoints:
        scale_w, scale_h = gw * patch_size / W, gh * patch_size / H
        dist = abs(scale_w - scale_h)
        if dist < min_dist:
            min_dist = dist; best_grid = (gw, gh)

    gw, gh = best_grid
    resized = img.resize((gw * patch_size, gh * patch_size))
    patches = []
    for y in range(gh):
        for x in range(gw):
            box = (x * patch_size, y * patch_size, (x+1)*patch_size, (y
                +1)*patch_size)
            patches.append(resized.crop(box))

    thumbnail = img.resize((patch_size, patch_size))
    return patches, thumbnail
```

28 Worked Napkin Problems in Vision

[Conv FLOPs · ViT attention scaling · VLM token budgets · 3DGS VRAM · Edge latency · ImageNet cluster math]

THE VISION NAPKIN RECIPE

1 Write down dimensions (H, W, C, P) . 2 Conv FLOPs $= 2 \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot K^2 \cdot H_0 W_0$. 3 ViT tokens $N = (H/P)(W/P)$; Self-Attention FLOPs $= 4ND^2 + 2N^2D$. 4 Compare against hardware roofline (compute vs bandwidth).

INTERVIEW Q&A | CONVOLUTION & TRANSFORMER SIZING

Q94. Calculate exact FLOPs for a 3×3 Conv2D layer with $C_{\text{in}} = 256, C_{\text{out}} = 512$, input 28×28 , stride 1, pad 1.
Output spatial size: $H_0 = W_0 = 28$.

$$\begin{aligned} \text{FLOPs} &= 2 \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot K^2 \cdot H_0 \cdot W_0 \\ &= 2 \cdot 256 \cdot 512 \cdot 9 \cdot 784 \approx 1.85 \text{ GFLOPs} \end{aligned}$$

If replaced with Depthwise-Separable conv: Depthwise: $2 \cdot 256 \cdot 1 \cdot 9 \cdot 784 \approx 3.6$ MFLOPs. Pointwise: $2 \cdot 256 \cdot 512 \cdot 1 \cdot 784 \approx 205$ MFLOPs. Total ≈ 209 MFLOPs ($8.8\times$ reduction).

Q95. ViT-Base ($D = 768$): How do Self-Attention FLOPs scale from 224×224 to 448×448 ($P = 16$)?

At 224^2 : $N_1 = (224/16)^2 = 14^2 = 196$ tokens. At 448^2 : $N_2 = (448/16)^2 = 28^2 = 784$ tokens ($4\times$ increase in tokens). Projections cost ($4ND^2$): grows linearly with tokens ($4\times$). Score matmul cost ($2N^2D$): grows quadratically ($4^2 = 16\times$). For $N = 196$: $2N^2D = 2(196^2)(768) \approx 59$ MFLOPs. For $N = 784$: $2N^2D = 2(784^2)(768) \approx 944$ MFLOPs.

Q96. What is the cumulative Receptive Field of 4 conv layers with kernels $[3, 3, 3, 3]$ and strides $[1, 2, 1, 2]$?

$J_0 = 1, RF_0 = 1$. Layer 1 ($K = 3, S = 1$): $RF_1 = 1 + (3 - 1) \cdot 1 = 3$; $J_1 = 1 \cdot 1 = 1$. Layer 2 ($K = 3, S = 2$): $RF_2 = 3 + (3 - 1) \cdot 1 = 5$; $J_2 = 1 \cdot 2 = 2$. Layer 3 ($K = 3, S = 1$): $RF_3 = 5 + (3 - 1) \cdot 2 = 9$; $J_3 = 2 \cdot 1 = 2$. Layer 4 ($K = 3, S = 2$): $RF_4 = 9 + (3 - 1) \cdot 2 = 13$; $J_4 = 2 \cdot 2 = 4$. Final receptive field is 13×13 pixels.

INTERVIEW Q&A | MULTIMODAL, 3D & EDGE MATH

Q97. A VLM processes a high-res image with AnyRes (4×336^2 crops + 1 thumbnail). Token budget?

Base ViT patch size is $P = 14$. Each 336×336 tile yields $(336/14)^2 = 24 \times 24 = 576$ patch tokens. Total tiles $= 4$ crops + 1 thumbnail $= 5$ images. Total raw visual tokens $= 5 \times 576 = 2,880$ tokens. With

2×2 pixel-shuffle downsampling: $2,880/4 = 720$ tokens in LLM context.

Q98. VRAM footprint of a 3D Gaussian Splatting scene with 3,000,000 Gaussians?

Each Gaussian requires 59 FP32 values (236 bytes):

$$\text{Weights} = 3,000,000 \times 236 \text{ bytes} \approx 708 \text{ MB}$$

Rasterizer tile buffers and sorting indices add ≈ 250 MB. Easily fits on an 8 GB consumer GPU or mobile phone memory.

Q99. Uncompressed 1080p60 camera stream bandwidth vs H.264 vs VAE latents?

Raw RGB: $1920 \times 1080 \times 3 \text{ bytes} \times 60 \text{ FPS} \approx 373 \text{ MB/s}$ (2.98 Gbps).
H.264 stream (15 Mbps): $\approx 1.87 \text{ MB/s}$ (200 $\times$ compression). Spatial VAE ($f = 8$, 4 channels, fp16): $\frac{1920}{8} \times \frac{1080}{8} \times 4 \times 2 \text{ bytes} \times 60 \approx 15.5 \text{ MB/s}$.

Q100. How long does it take to train ResNet-50 for 90 epochs on ImageNet-1K on 8x A100s?

ImageNet-1K contains 1.28×10^6 training images. ResNet-50 costs ≈ 4.1 GFLOPs forward $\Rightarrow \approx 12.3$ GFLOPs forward+backward per image. Total training compute for 90 epochs:

$$\text{Compute} \approx 90 \times (1.28 \cdot 10^6) \times (12.3 \cdot 10^9) \approx 1.42 \text{ ExaFLOPs}$$

One A100 provides 312 TFLOP/s FP16 Tensor Core compute. At 40% MFU: $8 \times (312 \times 10^{12}) \times 0.40 \approx 1.0 \times 10^{15}$ FLOP/s = 1 PFLOP/s.

$$\text{Training Time} = \frac{1.42 \times 10^{18}}{1.0 \times 10^{15}} \approx 1,420 \text{ seconds} \approx 23.6 \text{ minutes}$$

This is a compute floor: ResNet-50 rarely exceeds 40% MFU, and JPEG decoding/augmentation usually becomes the bottleneck first, so real 8x A100 runs (DALI, FFCV) take roughly 1–3 hours. Say both numbers.

29 Vision Systems: Hardware Scaling & Napkin Math II

[Video attention complexity · patch size quadratic scaling · lr batch scaling · distributed CLIP all-gather · NPU TOPS]

QUICK FORMULAS FOR VISION SYSTEM ARCHITECTS

Metric	Exact Formula	Typical Magnitude
ViT Patch Shrinkage	$N = \frac{HW}{p^2}; \text{Attn} \propto P^{-4}$	$P = 16 \rightarrow 8 \Rightarrow 16 \times$ Attention FLOPs!
Video Attn Savings	$\frac{\mathcal{O}(LS^2 + SL^2)}{\mathcal{O}((LS)^2)} \approx \frac{1}{L} + \frac{1}{S}$	Factorized attention saves $\sim 15 - 20 \times$ VRAM
Linear (SGD)	$\text{LR } \eta = \eta_0 \times \frac{B}{B_0}$	Scales with batch size B ; 5-ep warmup
Sqrt (AdamW)	$\text{LR } \eta = \eta_0 \times \sqrt{\frac{B}{B_0}}$	Used in large-scale ViT and diffusion
NPU Frame Time	$T \approx \frac{\text{FLOPs}}{\text{TOPS} \times \text{Util}}$	Edge NPU utilization typically 25% – 40%

INTERVIEW Q&A | VIDEO ATTENTION & PATCH RESOLUTION COMPLEXITY

Q101. Derive the memory and compute savings of factorized spatial-temporal attention vs joint 3D attention in video transformers.

Consider a video clip with T temporal frames and spatial resolution $H \times W$. After patchification, let spatial tokens per frame be $S = \frac{HW}{p^2}$ and temporal tokens be $L = T$:

- **Joint 3D Space-Time Attention:** Every token attends to all tokens across all frames. Total sequence length $N_{\text{total}} = L \cdot S$. The attention matrix size is:

$$\text{Memory}_{\text{joint}} = \mathcal{O}((L \cdot S)^2) = \mathcal{O}(L^2 S^2)$$

For a 16-frame clip with $14 \times 14 = 196$ patches per frame: $N_{\text{total}} = 16 \times 196 = 3136 \Rightarrow \text{Tokens}^2 \approx 9.83 \times 10^6$ pairs.

- **Factorized Spatial-Temporal Attention (TimeSformer / Sora):** Decomposes attention into (1) spatial attention within each frame, followed by (2) temporal attention across corresponding spatial locations:

$$\text{Memory}_{\text{factorized}} = L \cdot \mathcal{O}(S^2) + S \cdot \mathcal{O}(L^2) = \mathcal{O}(LS^2 + SL^2)$$

- **Savings Ratio:**

$$\frac{\text{Factorized}}{\text{Joint}} = \frac{LS^2 + SL^2}{L^2 S^2} = \frac{1}{L} + \frac{1}{S}$$

$$\text{For } L = 16, S = 196: \frac{1}{16} + \frac{1}{196} \approx 0.0625 + 0.0051 \approx 0.0676 \Rightarrow \sim 15 \times \text{ memory reduction!}$$

Q102. What happens mathematically to compute, memory, and receptive field when patch size is halved from $P = 16$ to $P = 8$ in ViT?

Given image size $H \times W$:

1. **Token Count:** $N = \frac{HW}{p^2}$. Halving P increases token sequence length by $4 \times$.
2. **Self-Attention FLOPs:** $4Nd^2 + 2N^2d$. The quadratic term $2N^2d$ scales by $(4)^2 = 16 \times$!
3. **KV Cache / Activation Memory:** Attention score matrix requires $16 \times$ higher intermediate activation memory.
4. **Spatial Representation:** Patch embedding receptive field shrinks from 16×16 to 8×8 , capturing fine-grained edges and small objects at the expense of massive GPU compute.

INTERVIEW Q&A | DISTRIBUTED ALL-GATHER COMMUNICATION & NPU LATENCY

Q103. Calculate the exact network bandwidth and transmission time for All-Gather in large-batch CLIP pretraining.

Consider training CLIP with total batch size $B = 65,536$ distributed across $P = 64$ GPUs (1024 samples/GPU), embedding dimension $D = 768$ in FP16 (2 bytes/float):

- **Local Tensor Size:** Each GPU holds an image embedding matrix of size:

$$M_{\text{local}} = 1024 \times 768 \times 2 \text{ bytes} = 1.57 \text{ MB}$$

- **All-Gather Data Transferred:** In a ring all-gather, each GPU transmits and receives $\frac{P-1}{P} \times \text{Total Data}$:

$$\text{Total Data} = 65,536 \times 768 \times 2 \text{ bytes} = 100.66 \text{ MB}$$

$$\text{Data sent per GPU} = \frac{63}{64} \times 100.66 \text{ MB} \approx 99.09 \text{ MB.}$$

- **Transfer Time over 400 Gbps (50 GB/s) InfiniBand:**

$$t_{\text{comm}} = \frac{99.09 \text{ MB}}{50 \text{ GB/s}} \approx 1.98 \text{ ms}$$

Because both image and text embeddings must be gathered, communication overhead is ~ 4 ms per training step.

Q104. How do you calculate real-world inference frame rate on an edge NPU given theoretical TOPS?

The Trap: A 26 TOPS NPU does not deliver 26 trillion operations per second on vision models!

- **Effective Formula:**

$$\text{FPS}_{\text{real}} = \frac{\text{NPU TOPS} \times 10^{12} \times \eta_{\text{hardware}}}{\text{Model GFLOPs} \times 10^9}$$

where hardware efficiency η_{hardware} depends on operator memory bandwidth (typically 25% – 40%).

- **Example:** YOLOv8s (28.6 GFLOPs) on an INT8 26 TOPS edge processor with realistic $\eta = 30\%$:

$$\text{FPS}_{\text{real}} = \frac{26 \times 10^{12} \times 0.30}{28.6 \times 10^9} \approx \frac{7,800}{28.6} \approx 272 \text{ FPS}$$

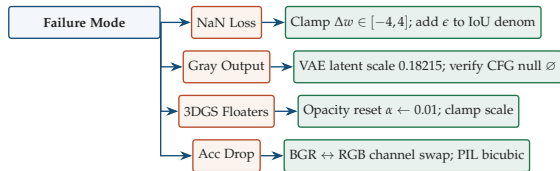
$$\text{Frame latency } t_{\text{frame}} = \frac{1000}{272} \approx 3.67 \text{ ms.}$$

SCALING TRAPS IN VISION INFRASTRUCTURE

- **Linear LR scaling without warmup:** Goyal et al. scale SGD's LR linearly with batch (0.1 per 256 images) but need a 5-epoch warmup; without it, large-batch runs diverge early. Beyond $\sim 8k$ images per batch, linear scaling breaks down and LARS/LAMB are used.
- **NPU Memory Bandwidth Bottlenecks:** High TOPS cannot compensate for low LPDDR bandwidth ($< 50 \text{ GB/s}$). High-resolution inputs saturate memory buses rather than compute units, capping real frame rates far below theoretical TOPS.

30 Debugging Playbooks: Vision Failure Modes

[NaN bounding box losses · ViT convergence failure · Diffusion gray output · 3DGS needle floaters · VLM repetition]



INTERVIEW Q&A | PRODUCTION DEBUGGING SCENARIOS

Q105. Detection loss suddenly becomes NaN at step 2,400. What is broken and how do you triage?

Root Causes:

- **Zero-Area Box / Division by Zero:** In GIoU or CIoU, if predicted or ground-truth coordinates satisfy $x_2 \leq x_1$ or $y_2 \leq y_1$, box area becomes 0 or negative. The enclosing convex hull denominator $|C|$ produces division by zero ($0/0 \rightarrow \text{NaN}$).
 - **Extreme Log Ratio:** If using anchor delta parametrization $\Delta w = \log(w/w_a)$, an unconstrained linear head predicting large positive values (e.g. +89) causes $\exp(\Delta w)$ to overflow FP16 max (65,504).
- The Fix:** Enforce strict coordinate ordering before loss: $x_1 = \min(x_1, x_2)$, $x_2 = \max(x_1, x_2) + \epsilon$. Clamp box width/height outputs: $\Delta w = \text{clamp}(\Delta w, \min = -4.0, \max = 4.0)$. Add epsilon ($\epsilon = 10^{-7}$) to all IoU denominators.

Q106. A ViT model fails to train from scratch on a 50k image dataset (loss plateaus at chance).

Diagnosis: Vision Transformers lack 2D local translation and neighborhood inductive biases. On small datasets ($< 100k$ images), random gradient steps in unconstrained self-attention space fail to find spatial structure. **The Playbook:**

1. **Initialize from Pretrained Checkpoints:** Fine-tune from MAE, DINOv2, or ImageNet-21k checkpoints rather than random initialization.
2. **Inject Heavy Regularization:** If training from scratch is mandatory, implement DeiT's recipe: RandAugment ($N = 2, M = 9$), Mixup ($\alpha = 0.8$), CutMix ($\alpha = 1.0$), and Stochastic Depth ($p = 0.1$).
3. **Switch to Hybrid/CNN:** Replace ViT with ConvNeXt-Tiny or Swin-T, whose built-in inductive priors converge reliably on small datasets.

Q107. Latent Diffusion generates solid grey or noisy static images despite low training loss.

Diagnosis:

1. **Missing VAE Latent Scale:** SD VAE latents have std ≈ 5.5 ($1/0.18215$). SD requires scaling latents by 0.18215 (SDXL: 0.13025) to enforce unit variance before adding Gaussian noise. Forgetting this leaves signal-to-noise ratio completely mismatched.
2. **Inverted Timestep Schedule:** Inverting the noise schedule (feeding $t = 0$ as pure noise instead of clean image).
3. **CFG Dropout Bug:** Dropping conditions during training without using the exact same null prompt embedding $\emptyset$ at test time.

Q108. 3D Gaussian Splatting scene produces needle floaters and blurry foreground artifacts.

Diagnosis: Needle floaters stem from Gaussians with one elongated covariance axis ($s_x \gg s_y, s_z$) in unobserved camera frustums. Blurry foreground stems from failure of adaptive density control to clone under-reconstructed Gaussians. **The Fix:** Clamp maximum spatial scale: $s = \min(s, s_{\max})$. Enforce **Periodic Opacity Reset:** Every 3,000 steps, force all Gaussian opacities to near-zero ($\alpha_i \leftarrow 0.01$). Valid surface Gaussians quickly regain opacity from photometric gradients; spurious floaters in empty space remain low and are culled by $\alpha < 0.005$ thresholding.

Q109. A VLM repeats generated output tokens in an infinite loop during image reasoning.

Diagnosis:

1. **Visual Attention Sink:** If the projector maps visual tokens to an embedding scale significantly smaller or larger than text token embeddings, attention weights in the LLM backbone saturate on initial visual tokens.
2. **Missing End-of-Sequence (EOS) Supervision:** Pretraining data omitted $\langle \text{im_end} \rangle$ or $\langle /s \rangle$ tokens, teaching the LLM to continue emitting tokens forever.

The Fix: Add LayerNorm at projector exit to equalize visual and text embedding variance; enforce repetition penalties during decoding; audit presence of EOS tokens in instruction tuning targets.

Q110. Validation accuracy drops 25% on deployed model despite zero

change in weights.

Triage Checklist:

1. **Color Channel Swap:** Check if deployed inference feeds OpenCV BGR into a model trained on Torchvision RGB.
2. **Resizing Interpolation:** Training used PIL bicubic with antialiasing; deployed engine used `INTER_NEAREST` or `INTER_LINEAR`.
3. **Normalization Mean/Std:** Input was scaled to $[0, 1]$ but omitted ImageNet subtract mean and divide std.

31 Rapid-Fire Q&A and Vision Glossary

[Rapid interview drills · core definitions · precise terminology]

31.1 Rapid-Fire Drills (Answer in under 5 seconds)

Why use 1×1 convs in ResNet bottleneck? → Changes channel dimension cheaply without changing spatial resolution.

What does RoIAlign do that RoIPool didn't? → Eliminates coordinate quantization using 4-point bilinear interpolation.

Why do ViTs need positional embeddings? → Self-attention is permutation-equivariant; without PosEnc, patches have no spatial order.

What is the FLOP cost of a 2D convolution? → $2 \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot K^2 \cdot H_0 W_0$.

Why does SigLIP scale better than CLIP? → Pairwise sigmoid loss needs no global softmax, so logits are computed in $b \times b$ chunks with a ring exchange instead of a full $B \times B$ matrix.

What is the patch count for 384×384 at $P = 16$? → $(384/16)^2 = 24^2 = 576$ tokens.

What is the spatial compression of standard SD VAE? → $8 \times$ in height and width ($64 \times$ area compression).

What target does Flow Matching predict? → Constant velocity vector field $v_t = x_1 - x_0$.

What is transmittance in NeRF? → Probability that a light ray travels from t_n to t without colliding with particles.

How many floats represent a 3D Gaussian in 3DGS? → 59 floats: 3 position, 3 scale, 4 quaternion, 1 opacity, 48 spherical harmonics.

What is Focal Loss's focusing parameter? → γ (typically $\gamma = 2$), which exponentially downweights easy negatives.

Why does MAE mask 75% of patches? → Images have high spatial redundancy; high masking prevents trivial pixel interpolation.

What does DINO self-attention map capture? → Unsupervised semantic segmentation masks of foreground objects.

What causes object hallucination in VLMs? → Language priors from text pretraining overpowering visual token attention.

How is Panoptic Quality calculated? → $PQ = SQ \times RQ$ (Segmentation Quality $\times$ Recognition Quality).

What is the difference between SIFT and ORB? → SIFT uses floating-point DoG histograms; ORB uses FAST corners and binary BRIEF strings.

What is the epipolar line equation? → $l' = Fx$, where F is the Fundamental Matrix.

Why is the linear bottleneck in MobileNetV2 linear? → ReLU non-linearities in low-dimensional manifolds collapse and zero out features.

What is the formula for the Canny gradient direction? → $\theta = \arctan(G_y/G_x)$, quantized into $0^\circ, 45^\circ, 90^\circ, 135^\circ$.

What is the purpose of adaLN-Zero in DiT? → Initializes scale gating to zero so every block acts as identity at step 0.

31.2 The Vision Glossary

Anchor Box Pre-defined candidate bounding box with fixed scale and aspect ratio used in detectors like Faster R-CNN and YOLOv3.

ASPP Atrous Spatial Pyramid Pooling; parallel atrous convolutions with diverse dilation rates capturing multi-scale context.

BEV (Bird's-Eye-View) Top-down 2D coordinate representation of 3D scenes used in autonomous driving perception.

CFG (Classifier-Free Guidance) Diffusion sampling technique extrapolating conditional over unconditional predictions: $\epsilon_{\emptyset} + s(\epsilon_c - \epsilon_{\emptyset})$.

Deformable Conv Convolution where 2D sampling grid offsets are predicted dynamically from input features.

DLT (Direct Linear Transformation) Linear algorithm for solving projective camera and homography matrices from point correspondences.

Epipolar Plane 3D plane formed by a 3D scene point and the two camera optical centers.

FPN (Feature Pyramid Network) Top-down architecture with lateral skip connections combining multi-scale semantic and spatial features.

GIoU (Generalized IoU) IoU metric penalized by normalized empty space within the smallest enclosing bounding box.

Homography 3×3 projective transformation mapping coplanar 3D points between two camera views: $x' \sim Hx$.

Inverted Residual Block structure expanding channels with 1×1 conv before 3×3 depthwise conv, ending with linear bottleneck.

LPIPS Learned Perceptual Image Patch Similarity; distance between deep feature activations from a pretrained VGG network.

NMS (Non-Maximum Suppression) Greedy algorithm pruning overlapping bounding boxes based on IoU thresholding.

Patchify Splitting a 2D image into non-overlapping grid tiles and linearly projecting them to 1D token embeddings.

RoIAlign Bilinear interpolation sampling inside region of interest bins to preserve floating-point spatial alignment.

Spherical Harmonics (SH) Orthogonal basis functions on the 2D sphere modeling view-dependent radiance in NeRF and 3DGS.

Swin Window Attention Restricting self-attention to localized $M \times M$ patch windows with linear complexity $\mathcal{O}(N)$.

Transmittance Accumulation of transparency along a camera ray in volumetric rendering: $T(t) = \exp(-\int \sigma ds)$.

VQ-VAE Vector Quantized Variational Autoencoder mapping continuous image latents to discrete codebook indices.

32 High-Velocity Vision Interview Drill

[20 rapid-fire technical questions · flashcards · edge cases · instant answers]

INTERVIEW Q&A | RAPID-FIRE ARCHITECTURE & MECHANICS DRILLS

Why do ViTs fail when trained from scratch on small datasets without pretraining? → ViTs lack the inductive biases of convolutions (translation equivariance and local spatial locality); they must learn spatial relationships from scratch via massive data.

How do you resize learned 2D positional embeddings when fine-tuning a ViT at higher resolution? → Bicubic 2D spatial interpolation on the $(H/P) \times (W/P)$ position grid, leaving the [CLS] embedding unchanged.

What is the purpose of the hard distillation token in DeiT? → It interacts with patch tokens and [CLS] via self-attention, but its loss is trained exclusively against the hard class labels predicted by a teacher ConvNet (RegNet).

Why does Soft-NMS outperform standard Hard-NMS in dense crowd scenes? → Hard-NMS instantly zeros the score of overlapping boxes, destroying true overlapping objects; Soft-NMS smoothly decays the score using a Gaussian or linear penalty: $s_i \leftarrow s_i \exp(-\frac{\text{IoU}_{ij}^2}{\sigma})$.

Why does ASPP in DeepLab use atrous (dilated) convolutions instead of pooling? → Dilated convolutions expand the receptive field to capture multi-scale context without downsampling spatial resolution or losing fine-grained edge localization.

What is the Degenerate Grid trap in large atrous convolution dilation rates? → When dilation rate r approaches feature map size W , a 3×3 kernel samples outside bounds or only isolated pixels, degenerating into a trivial 1×1 convolution.

Why can't BatchNorm be used directly in Vision Transformers? → Batch statistics fluctuate excessively across sequence lengths, and ViT token representations undergo covariate shifts along sequence dimensions; LayerNorm normalizes across feature channels per token independently.

What is the difference between Transposed Convolution and Bilinear Upsampling? → Transposed convolutions learn upsampling weights but suffer from "checkerboard artifacts" when stride does not divide kernel size; Bilinear upsampling is non-parametric, smooth, and artifact-free.

Why does CLIP require a learned temperature parameter τ ? → Cosine similarities lie in $[-1, 1]$; with 32k negatives the softmax over such logits is almost uniform (the positive gets at most $e^1 / (e^1 + 32767 e^{-1}) \approx 0.02\%$). Dividing by $\tau \approx 0.07$ stretches logits to $[-14, 14]$ so the positive can win.

What is the All-Gather gradient bottleneck in distributed CLIP? → `all_gather` is not differentiable, so gathered negatives carry no gradient. Fix: replace your own rank's slice with the local tensor (keeps its gradient), or use a differentiable gather and compute only the local rows of the logit matrix (OpenCLIP `local_loss`).

Why does Min-SNR γ loss weighting improve diffusion model convergence? → Diffusion loss at small t has massive SNR and over-supervises high-frequency noise; clamping loss weight $\min(\text{SNR}(t), \gamma)$ balances gradient contributions across all timesteps.

What is the difference between DDPM and DDIM sampling? → DDPM is a stochastic Markov chain requiring 1000 steps; DDIM defines an equivalent non-Markovian process whose deterministic ODE allows skipping steps in 20–50 iterations.

Why does Flow Matching use straight trajectories instead of curved

Brownian paths? → Straight ODE paths minimize displacement length between noise and data, resulting in velocity fields that are nearly constant in time, enabling 1-step or 4-step Euler integration.

Why does NeRF require Hierarchical Volume Sampling (Coarse and Fine networks)? → Uniform ray sampling wastes 90% of MLP evaluations in empty space or occluded regions; the coarse network generates a probability density function along the ray to concentrate fine samples around surfaces.

Why does 3DGS use Spherical Harmonics instead of a tiny MLP for color? → Spherical Harmonics evaluate in closed form via direct polynomial multiplication ($< 0.1 \mu s$ on CUDA), whereas evaluating an MLP per Gaussian would bottleneck real-time 100+ FPS rendering.

How does ByteTrack track occluded and low-confidence detections? → Instead of discarding boxes with confidence < 0.5 , ByteTrack matches high-confidence boxes first, then runs a second association matching low-confidence boxes to unmatched tracklets.

What is the difference between Post-Training Quantization (PTQ) and QAT? → PTQ quantizes frozen FP32 weights using a calibration dataset without backpropagation ($\sim$ minutes); QAT simulates fake-quantization during retraining with straight-through estimator ($\sim$ days).

What is the aperture problem in optical flow? → Through a small aperture (local window), motion parallel to an edge cannot be detected; only normal motion perpendicular to the gradient is measurable.

Why does Mask2Former unify semantic, instance, and panoptic segmentation? → It treats all segmentation tasks as binary mask prediction with query-based cross-attention constrained inside predicted mask regions (Masked Attention).

What is the difference between Epipolar Line and Epipole? → The epipole is the projection of the other camera center onto the image plane; the epipolar line is the intersection of the epipolar plane with the image plane, upon which all matching points must lie.

CRUCIAL INTERVIEW TRAPS & FAILURE MODES

- **ViT Quadratic Complexity Assumption:** Candidates often say "ViT is always $\mathcal{O}(N^2)$ ". FlashAttention still does $\mathcal{O}(N^2)$ FLOPs but never materializes the $N \times N$ matrix, so memory is $\mathcal{O}(N)$ and it runs several times faster; only windowed (Swin) or linear attention changes the FLOP scaling.
- **Assuming Diffusion Directly Predicts the Clean Image x_0 :** Predicting noise ϵ_t or velocity v_t is dramatically more numerically stable than predicting clean image x_0 , because at large t , x_0 has near-zero signal-to-noise ratio and predicting it causes blurry mean-image collapse.
- **Overlooking Bounding Box Coordinate Normalization:** Feeding absolute pixel coordinates (x, y, w, h) into loss functions rather than normalized $[0, 1]$ coordinates breaks scale-invariance across different image sizes.

33 Master Vision Formula Sheet

[Convolution · receptive field · attention FLOPs · geometry · detection · diffusion · 3DGS]

Mechanism	Mathematical Formulation & Invariants
Conv2D Output	$O = \lfloor \frac{I-K+2P}{S} \rfloor + 1$ (dilation) $d \implies K' = K + (K-1)(d-1)$
Conv2D FLOPs	$\text{FLOPs} = 2 \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot K^2 \cdot H_{\text{out}} \cdot W_{\text{out}}$
Receptive Field	$RF_l = RF_{l-1} + (K_l - 1) \cdot J_{l-1}, \quad J_l = J_{l-1} \cdot S_l$
Depthwise Ratio	$\frac{\text{FLOPs}_{\text{sep}}}{\text{FLOPs}_{\text{std}}} = \frac{1}{C_{\text{out}}} + \frac{1}{K^2} \approx \frac{1}{9}$ (for $3 \times 3, C_{\text{out}} \gg 1$)
Standard ViT Attn	$\Omega(\text{MSA}) = 4ND^2 + 2N^2D, \quad \Omega(\text{MLP}) = 8ND^2$
Swin Window Attn	$\Omega(\text{W-MSA}) = 4ND^2 + 2M^2ND$ (linear in $N, M = 7$)
Harris Response	$R = \det(M) - k \cdot (\text{Tr}(M))^2 = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$
Optical Flow	$I_x u + I_y v + I_t = 0 \iff \nabla^T I \mathbf{v} + I_t = 0$ (Aperture)
Epipolar Line	$x_1^T F x_1 = 0$ (uncalibrated F), $\hat{x}_2^T E \hat{x}_1 = 0$ (E)
RANSAC Count	$N = \frac{\ln(1-p)}{\ln(1-(1-\epsilon)^s)}$ (sample s , outlier ratio ϵ)
Focal Loss	$\text{FL}(p_t) = -\alpha_t (1-p_t)^\gamma \log(p_t)$ ($\gamma = 2.0, \alpha_t = 0.25$)
CIoU Loss	$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{e^{\frac{b^2}{b^2_{\text{gt}}}}}{c^2} + \alpha v$ $v = \frac{4}{\pi^2} \left(\arctan \frac{w^{\text{gt}}}{h^{\text{gt}}} - \arctan \frac{w}{h} \right)^2$
TAL Metric	$t = s^\alpha \times \text{IoU}^\beta$ ($\alpha = 0.5, \beta = 6.0$ in YOLOv8/v11)
CLIP InfoNCE	$\mathcal{L} = -\frac{1}{2B} \sum_{i=1}^B \left[\log \frac{e^{S_{ii}/\tau}}{\sum_j e^{S_{ij}/\tau}} + \log \frac{e^{S_{ji}/\tau}}{\sum_j e^{S_{ji}/\tau}} \right]$
SigLIP Loss	$\mathcal{L} = -\frac{1}{B} \sum_{i,j} \log \sigma(y_{ij}(I_i^T T_j \cdot e^t + b))$, $y_{ij} \in \{\pm 1\}$
DDPM Noising	$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$, $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$
CFG Score	$\tilde{\epsilon}_\theta(x_t, c) = \epsilon_\theta(x_t, \emptyset) + s \cdot (\epsilon_\theta(x_t, c) - \epsilon_\theta(x_t, \emptyset))$
Rectified Flow	$x_t = (1-t)x_0 + tx_1$, $\frac{dx_t}{dt} = v_t = x_1 - x_0$
NeRF Quadrature	$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t) \sigma(\mathbf{r}(t)) \mathbf{c}(\mathbf{r}(t), \mathbf{d}) dt$ $T(t) = \exp\left(-\int_{t_n}^t \sigma(\mathbf{r}(s)) ds\right)$
3DGS Projected Cov	$\mathbf{\Sigma} = \mathbf{JW}\mathbf{\Sigma}^T \mathbf{J}^T + 0.3 \cdot \mathbf{I}_{2 \times 2}$, $\mathbf{\Sigma} = \text{RSS}^T \mathbf{R}^T$
Panoptic Quality	$\text{PQ} = \frac{\sum_{\text{TP}} \text{IoU}}{ \text{TP} + \frac{1}{2} \text{FP} + \frac{1}{2} \text{FN} } = \text{SQ} \times \text{RQ}$
FID Score	$\text{FID} = \ \mu_r - \mu_g\ _2^2 + \text{Tr}(\mathbf{\Sigma}_r + \mathbf{\Sigma}_g - 2(\mathbf{\Sigma}_r \mathbf{\Sigma}_g)^{1/2})$
Conv-BN Folding	$W_{\text{fused}} = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} W$, $b_{\text{fused}} = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} (b - \mu) + \beta$

34 30 Seminal Vision Papers

[Foundational papers · Authors & Year · Breakthroughs · Paradigms]

- AlexNet** (Krizhevsky et al., 2012): Deep convolutional network with ReLU, Dropout, and GPU parallelization; shattered ImageNet error from 26% to 16%, launching deep learning.
- VGG** (Simonyan & Zisserman, 2014): Proved that deep stacks of small 3×3 convs outperform wide filters (7×7), establishing modern homogeneous ConvNet architecture.
- ResNet** (He et al., 2015): Residual identity shortcuts solving vanishing gradients up to 152 layers ($\mathcal{H}(x) = \mathcal{F}(x) + x$), winner of ImageNet, COCO, and ILSVRC 2015.
- U-Net** (Ronneberger et al., 2015): Symmetric encoder-decoder architecture with cross-skip concatenation preserving spatial details for dense pixel-level medical segmentation.
- Faster R-CNN** (Ren et al., 2015): First end-to-end deep detector; introduced Region Proposal Networks (RPN) sharing convolutional features with Fast R-CNN detector.
- Mask R-CNN** (He et al., 2017): RoIAlign with bilinear interpolation eliminating sub-pixel quantization; parallel FCN branch adding instance segmentation to object detection.
- RetinaNet** (Lin et al., 2017): Focal loss solving extreme 1000:1 foreground-background class imbalance, enabling single-stage detectors to match two-stage accuracy.
- MobileNetV2** (Sandler et al., 2018): Inverted residuals and linear bottlenecks preserving manifold capacity in low-dimensional space for mobile inference.
- YOLO Family** (Redmon 2016 / Jocher 2023): Real-time single-stage detection evolving from grid regression (v1) to CSPNet (v5) and anchor-free TAL + DFL heads (v8/v11).
- DETR** (Carion et al., 2020): Reformulated object detection as direct set prediction using Transformers and Hungarian bipartite matching, completely eliminating hand-crafted NMS.
- ViT** (Dosovitskiy et al., 2020): Proved standard Transformer applied directly to sequences of 16×16 image patches attains SOTA ImageNet accuracy when pretrained on large data (JFT-300M).
- Deformable DETR** (Zhu et al., 2020): Replaced global attention with

multi-scale deformable attention sampling $K = 4$ sparse points per query, accelerating convergence by $10 \times$.

- Swin Transformer** (Liu et al., 2021): Shifted Window self-attention achieving linear computational complexity $\mathcal{O}(N)$ and hierarchical multi-scale representations for dense vision.
- SimCLR** (Chen et al., 2020): Contrastive self-supervised learning with normalized InfoNCE loss, stochastic data augmentations, and non-linear projection MLP heads.
- MAE** (He et al., 2021): Masked Autoencoders masking 75% of visual patches with an asymmetric encoder-decoder, demonstrating that autoencoding is scalable self-supervision for vision.
- DINO / DINOv2** (Caron 2021 / Oquab 2023): Self-distillation without labels via centering and sharpening; produces emergent semantic part segmentations and foundation visual representations.
- CLIP** (Radford et al., 2021): Contrastive Language-Image Pretraining on 400M web image-text pairs; enables zero-shot classification via natural language prompting.
- SigLIP** (Zhai et al., 2023): Sigmoid pairwise loss eliminating all-gather softmax normalization across distributed GPUs, scaling dual-encoder training to million-scale batch sizes.
- ConvNeXt-V2** (Woo et al., 2023): Modernized pure ConvNet with 7×7 depthwise kernels and Global Response Normalization (GRN), outperforming Swin on ImageNet and COCO.
- DDPM** (Ho et al., 2020): Denoising Diffusion Probabilistic Models establishing connection between diffusion probabilistic models and score-based denoising score matching.
- Latent Diffusion / Stable Diffusion** (Rombach et al., 2022): Moved diffusion into perceptual $f = 8$ latent space of a pretrained VAE, reducing compute by orders of magnitude.
- DiT** (Peebles & Xie, 2023): Diffusion Transformer replacing U-Net with ViT and adaLN-Zero modulation, establishing compute scaling laws for visual generative models.
- Flow Matching / Rectified Flow** (Lipman 2022 / Liu 2022): Straight continuous probability trajectories predicting constant velocity $v_t = x_1 - x_0$, enabling 1–4 step generation.
- SAM / SAM 2** (Kirillov 2023 / Ravi 2024): Foundation promptable segmentation models; SA-1B dataset; streaming memory bank for real-time video object segmentation.
- NeRF** (Mildenhall et al., 2020): Neural Radiance Fields synthesizing photorealistic novel views via 5D coordinate MLPs and differentiable volumetric rendering.
- 3D Gaussian Splatting** (Kerbl et al., 2023): Replaced coordinate MLPs with explicit 3D Gaussians and tile-based rasterization, achieving photorealistic novel-view synthesis at > 100 FPS.
- LLaVA / LLaVA-NeXT** (Liu 2023 / Liu 2024): Visual instruction tuning using CLIP and LLaMA backbones; AnyRes dynamic grid slicing for high-resolution document parsing.
- Qwen2-VL** (Wang et al., 2024): Native dynamic resolution VLM with 2D/3D visual RoPE, eliminating aspect ratio distortion and fixed grid constraints.
- Flux.1** (Black Forest Labs, 2024): 12B parameter Flow Matching generative model with rectified flow ODEs and multimodal double-stream transformer blocks.
- Sora** (OpenAI, 2024): Spacetime diffusion transformer operating on 3D video patches, scaling video generation across diverse durations, resolutions, and aspect ratios.

35 References: The Vision Canon

Classical Geometry, Filtering & Flow (1, 1b)

Hartley & Zisserman 2004, *Multiple View Geometry in Computer Vision*. Cambridge Univ Press

Lowe 2004, *Distinctive Image Features from Scale-Invariant Keypoints* (SIFT). *IJCV* 60(2)

Rublee et al. 2011, *ORB: An efficient alternative to SIFT or SURF*. *ICCV* 2011

Lucas & Kanade 1981; Horn & Schunck 1981, *Determining Optical Flow*. *AI Journal*

Teed & Deng 2020, *RAFT: Recurrent All-Pairs Field Transforms for Optical Flow*.

2003.12039

Convolutions & Modern Backbones (2, 2b)

He et al. 2016, *Deep Residual Learning for Image Recognition* (ResNet). 1512.03385

Sandler et al. 2018, *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. 1801.04381

Ma et al. 2018, *ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture*.

1807.11164

Tan & Le 2019, *EfficientNet: Rethinking Model Scaling for CNNs*. 1905.11946

Liu et al. 2022, *A ConvNet for the 2020s* (ConvNeXt); Woo et al. 2023 (ConvNeXt-V2).

2201.03545, 2301.00808

Vision Transformers & Hierarchical ViTs (3, 4)

Dosovitskiy et al. 2020, *An Image is Worth 16x16 Words* (ViT). 2010.11929

Touvron et al. 2021, *Training data-efficient image transformers* (DeiT). 2012.12877

Liu et al. 2021, *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*.

2103.14030

Dehghani et al. 2023, *Scaling Vision Transformers to 22 Billion Parameters (ViT-22B)*. 2302.05442

Dehghani et al. 2023, *Patch n' Pack: NaViT, a Vision Transformer for any Aspect Ratio*. 2307.06304

Object Detection & Set Prediction (5, 5b, 6)

Ren et al. 2015, *Faster R-CNN*; He et al. 2017, *Mask R-CNN*. 1506.01497, 1703.06870

Lin et al. 2017, *Focal Loss for Dense Object Detection (RetinaNet)*. 1708.02002

Carion et al. 2020, *End-to-End Object Detection with Transformers (DETR)*. 2005.12872

Zhu et al. 2020, *Deformable DETR*; Zhang et al. 2022, *DINO-DETR*. 2010.04159, 2203.03605

Wang et al. 2024, *YOLOv10: Real-Time End-to-End Object Detection*. 2405.14458

Segmentation & Foundation Models (7, 7b)

Ronneberger et al. 2015, *U-Net: Convolutional Networks for Biomedical Segmentation*. 1505.04597

Chen et al. 2017, *Rethinking Atrous Convolution for Semantic Image Segmentation (DeepLabv3)*. 1706.05587

Cheng et al. 2022, *Masked-attention Mask Transformer for Universal Segmentation (Mask2Former)*. 2112.01527

Kirillov et al. 2023, *Segment Anything (SAM)*; Ravi et al. 2024 (SAM 2). 2304.02643, 2408.00714

Self-Supervised Learning & Dual Encoders (8, 8b, 9)

Chen et al. 2020, *A Simple Framework for Contrastive Learning (SimCLR)*; He 2020 (MoCo). 2002.05709, 1911.05722

He et al. 2021, *Masked Autoencoders Are Scalable Vision Learners (MAE)*. 2111.06377

Caron et al. 2021, *Emerging Properties in Self-Supervised ViTs (DINO)*; Oquab 2023 (DINOv2). 2104.14294, 2304.07193

Radford et al. 2021, *Learning Transferable Visual Models (CLIP)*. 2103.00020

Zhai et al. 2023, *Sigmoid Loss for Language-Image Pre-Training (SigLIP)*. 2303.15343

Generative Diffusion, DiT & Flow Matching (11, 12, 13)

Ho et al. 2020, *DDPM*; Song et al. 2020, *Score SDE*; Song et al. 2020, *DDIM*. 2006.11239, 2011.13456, 2010.02502

Karras et al. 2022, *Elucidating the Design Space of Diffusion (EDM)*. 2206.00364

Rombach et al. 2022, *High-Resolution Image Synthesis with Latent Diffusion Models*. 2112.10752

Peebles & Xie 2023, *Scalable Diffusion Models with Transformers (DiT)*. 2212.09748

Lipman et al. 2022, *Flow Matching for Generative Modeling*; Liu 2022 (Flow Straight and Fast). 2210.02747, 2209.03003

Tong et al. 2023, *Improving and Generalizing Flow-Based Generative Models with Minibatch OT (OT-CFM)*. 2302.00482

Esser et al. 2024, *Scaling Rectified Flow Transformers for High-Res Synthesis (SD3 MM-DiT)*. 2403.03206

Video Foundations, 3D & Multimodal LLMs (10, 10b, 14, 15, 15b)

Mildenhall et al. 2020, *NeRF*; Kerbl et al. 2023, *3D Gaussian Splatting*. 2003.08934, 2308.04079

Bertasius et al. 2021, *TimeSformer*; Brooks et al. 2024 (Sora Technical Report). 2102.05095, openai.com

Liu et al. 2023, *LLaVA*; Wang et al. 2024, *Qwen2-VL: To See the World More Clearly*. 2304.08485, 2409.12191

Li et al. 2023, *Evaluating Object Hallucination in Large Vision-Language Models (POPE)*. 2305.10355

Leng et al. 2024, *Mitigating Object Hallucination via Visual Contrastive Decoding (VCD)*. 2311.16922

Fayssse et al. 2024, *ColPali: Efficient Document Retrieval with Vision Language Models*. 2407.01449

Li et al. 2022, *BEVFormer: Learning BEV Representation with Spatiotemporal Transformers*. 2203.17270

Zhang et al. 2022, *ByteTrack: Multi-Object Tracking by Associating Every Detection Box*. 2110.06864

36 The 21-Day Vision Mastery Curriculum

[Foundations · Multimodal & Generative · Systems & Edge · Code-Blind · Mock Rubrics]

THE 3-PHASE REVISION ARCHITECTURE		
Phase	Days	Core Focus & Daily Target
Phase 1	Days 1–7	Foundations: Classical CV, CNNs, ViTs, Detection, SAM, SSL
Phase 2	Days 8–14	Generative & 3D: CLIP, VLMs, Diffusion, Flow, Video, 3DGS
Phase 3	Days 15–21	Systems & Drills: Metrics, Edge NPU, Code-Blind, Mock Interviews

Day-by-Day Execution Schedule

- ▶ **Days 1–2 (Classical & Convolutions):** Derive Harris response R and Hartley 8-point normalization; calculate RANSAC sample count ($N = 72$ for $\epsilon = 0.5$); compute Conv2D output shape and receptive field recurrence ($RF_i = RF_{i-1} + (K - 1)J$); derive depthwise separable FLOP ratio ($\sim 1/9$); explain ConvNeXt-V2 GRN.
- ▶ **Days 3–4 (ViTs & Single-Stage Detectors):** Derive ViT self-attention FLOPs ($4ND^2 + 2N^2D$); trace DeiT distillation token and Swin cyclic shift masking; derive Focal Loss gradient suppression ($10,000\times$ on easy negatives); compare RoIPool vs RoIAlign; analyze YOLOv8/v11 TAL metric ($s^{\text{th}} \cdot \text{IoU}^{\text{th}}$) and DFL distribution regression.

- ▶ **Days 5–6 (DETR & Segmentation):** Trace DETR Hungarian bipartite assignment; calculate Deformable Attention sampling complexity ($4MK$ vs N); analyze DINO-DETR contrastive denoising; compare DeepLabv3+ ASPP to Mask2Former masked attention; walk through SAM ViT-H windowed backbone and SAM 2 streaming video memory bank.
- ▶ **Day 7 (Self-Supervised Learning):** Prove InfoNCE lower bound on mutual information; analyze BYOL non-collapse dynamics; explain MAE 75% masking asymmetric speedup and DINOv2 KoLeo entropy regularization with register tokens.
- ▶ **Days 8–9 (CLIP & Multimodal LLMs):** Derive CLIP InfoNCE gradients and distributed All-Gather backward bottleneck; compare CLIP softmax vs SigLIP pairwise sigmoid loss; diagram LLaVA AnyRes grid slicing; analyze Qwen2-VL native 2D/3D RoPE; explain POPE splits and VCD contrastive decoding.
- ▶ **Days 10–11 (Diffusion Foundations & DiT):** Derive DDPM forward noising $x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon$; connect score matching to Tweedie's formula; analyze EDM continuous σ scaling and CFG burn; contrast pixel diffusion vs latent VAE ($f = 8$); trace DiT adaLN-Zero compute scaling laws.
- ▶ **Days 12–13 (Flow Matching & Video Models):** Derive straight probability trajectories and constant velocity objective $v_t = x_1 - x_0$; explain 2-Rectified Flow Reflowing; contrast Flux.1 double vs single stream blocks; factorize 3D attention into spatial-then-temporal operations; prove $15\times$ memory reduction; analyze Sora spacetime patch grids.
- ▶ **Day 14 (3D NeRF & Gaussian Splatting):** Derive NeRF volumetric quadrature rendering; prove 3D covariance positive semi-definiteness ($\Sigma = RSS^TR^T$); trace 3DGS tile rasterizer with 64-bit Radix sort and EWA Jacobian projection.
- ▶ **Days 15–16 (Benchmarks & Edge Deployment):** Calculate Panoptic Quality $PQ = SQ \times RQ$; derive FID 2-Wasserstein distance under Gaussian assumption; evaluate POPE Yes-ratio and MMBench circular eval; derive Conv-BN folding; explain TensorRT INT8 calibration via KL divergence minimization; trace ByteTrack secondary association.
- ▶ **Days 17–18 (System Design & Code-Blind):** Architect autonomous vehicle 3D perception (BEVFormer); design ColPali late-interaction visual document retrieval; write blind in < 5 minutes: Conv2D via im2col, ResNet Bottleneck, ViT PatchEmbed, 2D RoPE, InfoNCE loss, and Vectorized NMS.
- ▶ **Days 19–20 (Training Loops & Napkin Math):** Write pure PyTorch execution loops for SimCLR step, DDPM step, Rectified Flow step, and AnyRes image slicer; compute without calculator: Conv FLOPs, ViT quadratic memory, distributed CLIP network bandwidth, 3DGS VRAM footprint, and edge NPU FPS.
- ▶ **Day 21 (Final Rapid-Fire Drill):** Fire through all 40 rapid-fire flashcards aloud; time each answer strictly under 5 seconds.

INTERVIEW RED FLAGS CANDIDATES FAIL ON

- ▶ **Hand-Waving FLOP Calculations:** Saying "convolutions are heavy" without stating $2C_{\text{in}}C_{\text{out}}K^2HW$ or confusing FLOPs with MACs ($1 \text{ MAC} = 2 \text{ FLOPs}$).
- ▶ **VLM Token Naivety:** Assuming high-resolution images are simply resized; production VLMs tile images into multi-grid patches, generating 1000 – 4000 tokens per image.
- ▶ **Confusing NeRF with 3DGS:** Claiming 3DGS uses an MLP; 3DGS is an **explicit** point-based representation with closed-form rasterization, whereas NeRF is an **implicit** continuous neural field.